من الفكرة للكود: النسخة الموسعة

Omar Fahid • ShopMate AI • مراجعة تعليمية بتاريخ 21 أكتوبر 2026

شنو تزاد؟

هاد الجزء كيمشي مع الملفات والدوال الفعلية. كل مقطع كود فيه أرقام السطور من النسخة المحلية، ومن بعدو شرح بالدارجة. فالآخر كاين فهرس ملفات، وملحق فيه الدليل الأول ديال الأساسيات.

كيفاش تستعملو؟

بدا برحلة الدخول، من بعد الكاتالوغ والطلب، ومن بعد RAG والشات. فكل مقطع جاوب: شنو دخل؟ شنو تحققنا منو؟ فين قرينا أو كتبنا؟ شنو رجع؟ هاد الأسئلة كتحول القراية لفهم.

الحدود ديال هاد الوثيقة

الشرح كيغطي أجزاء التطبيق والـscripts، وكيشرح المقاطع المهمة بالتفصيل. ماشي تعليق مستقل على كل سطر تنسيق أو كل مكتبة خارجية. نسخة HTML المرافقة فيها ملفات المصدر المختارة كاملة ومرقمة باش تتابع أي سطر خارج المقتطفات.

المصدر والدليل

النسخة المحلية مربوطة بـgithub.com/omartkd7/Shopmates.Ai، وHEAD اللي تقرا هو 73187a0. فتح GitHub عبر أداة الويب فشل؛ ما كنأكدوش التطابق مع آخر نسخة بعيدة. هادي مراجعة كود، ماشي نتيجة اختبارات جديدة ولا إثبات تشغيل الإنتاج.

مهم للمناقشة

التعليق فالكود يقدر يكون قديم. التنفيذ هو اللي كنشرحوه. ملي كاين فرق أو حد معروف، كتلقاه مذكور باش ما تقدمش ضمان أكبر من الموجود.

الفهرس • 1

اختار القسم اللي محتاج. أرقام الصفحات كتخص الشرح الموسع؛ الدليل الأول كاين فالملحق.

من الفكرة للكود: النسخة الموسعة	1
01 • اللغة اللي مكتوب بها المشروع	6
02 • async و await و try/catch	7
03 • app.ts: ترتيب استقبال الطلب	8
04 • middleware: حراس قبل الخدمة	9
05 • Zod: التحقق وقت التشغيل	10
06 • register: خلق الحساب خطوة بخطوة	12
07 • JWT و إصدار زوج التوكنات	13
08 • login ثم refresh ثم logout	14
09 • models و migrations و associations	15
10 • GET products: البحث والتقسيم	17
11 • تعديل المنتجات والأصناف	19
12 • إنشاء طلب من REST	20
13 • قراية الطلب وإلغاؤه	22
14 • RAG: upload والـtransaction	23
15 • chunkText: تتبع المؤشر	25
16 • embedding ثم SQL similarity	26
17 • حذف وإعادة فهرسة الوثائق	27

الفهرس • 2

اختار القسم اللي محتاج. أرقام الصفحات كتخص الشرح الموسع؛ الدليل الأول كاين فالملحق.

18 • llm.ts: الاتصال بالمزود 28

19 • streamChat: جمع أجزاء الجواب 29

20 • agent.ts: بداية الدور 30

21 • agent.ts: السياق والتاريخ 31

22 • loop ديال الأدوات 32

23 • allowlist و Zod و dispatcher 33

24 • recap و pending confirmation 34

25 • executeConfirmedAction: الشراء الفعلي 35

26 • guardrails.ts بالتفصيل 36

27 • SSE: النص النهائي كيخرج بعد المراجعة 37

28 • AgentLog: شنو كيتسجل؟ 38

29 • MCP client والخادم المنفصل 39

30 • n8n والإشعار بعد commit 40

31 • React: state و props و hooks 41

32 • navigation و استرجاع الدخول 43

33 • authStore و tokenStorage 44

34 • Axios interceptors و refreshPromise 45

35 • الكاتالوغ فالموبايل 46

الفهرس • 3

اختار القسم اللي محتاج. أرقام الصفحات كتخص الشرح الموسع؛ الدليل الأول كاين فالملحق.

36 • streamAgentChat: فك أحداث SSE 47

37 • chat.tsx: النص والبطاقات 48

38 • orders و profile والشاشات الإضافية 49

39 • إدارة الكاتالوغ فالموبايل 50

40 • إدارة الوثائق والسجلات 51

41 • theme والمكونات والتصميم 52

42 • Dockerfile و docker-entrypoint 53

43 • seed و SQL و migrations 55

44 • package.json و CI و scripts 56

45 • قراءة test: arrange / act / assert 57

46 • كيفاش تتبع bug وتجاوب اللجنة 58

47 • قاموس ملفات المشروع 59

Model • Store 74

Model • User 75

Model • RefreshToken 76

Model • Category 77

Model • Product 78

Model • Customer 79

الفهرس • 4

اختار القسم اللي محتاج. أرقام الصفحات كتخص الشرح الموسع؛ الدليل الأول كاين فالملحق.

Model • Order	80
Model • OrderItem	81
Model • Conversation	82
Model • Message	83
Model • Document	84
Model • Chunk	85
Model • AgentLog	86
48 • قراءة migrations بالترتيب	87
49 • تمارين تطبيقية بأجوبة	95

01 • اللغة اللي مكتوب بها المشروع

TypeScript كيوصف البيانات، وJavaScript هو السلوك اللي كيتنفذ. TSX كيخلينا نكتب واجهة React مع الكود.

backend/src/models/Product.ts
mobile/src/components/Button.tsx

import و export

كيخلي ملفات أخرى تستعملها. المسار اللي كيبدا بنقطة نسبي. export من ملف آخر component كيدخل دالة أو import للملف؛ @/ فالموبايل alias مضبوط فإعدادات TypeScript.

function و let و const

كتستعمل لقيمة غادي نبدلوها بحال let. يقدر يتبدل const داخل object كيثبت الربط ديال الاسم بالقيمة، ولكن total. function كتجمع تعليمات قابلة لإعادة الاستعمال.

backend/src/models/Product.ts

```
 4  interface ProductData {
 5
 6
 7
 8   id: number;
 9    storeId: number;
10    categoryId: number | null;
11    name: string;
12    description?: string | null;
13    imageUrl?: string | null;
14    price: number | string;
15    sku?: string | null;
16    stock: number;
17    createdAt?: Date;
18    updatedAt?: Date;
19  }
```

شرح الأنواع

من DECIMAL كتعني الثمن يقدر يجي بأي واحد من هاد النوعين؛ price: number | string. نص string، عدد number. قيمة صريحة كتقول ما كايناش معلومة null. يقدر يرجع كنص. ? كتعني الخاصية اختيارية PostgreSQL.

as و ?. و ! و ??

هو authenticate كيقول للمحلل افترض موجود؛ req.user!. وما كيحققش المدخلات وقت التشغيل TypeScript كيوجه as وكيحافظ، undefined أو null اللي خاصو يضمن هادشي. ?. وصول آمن إلا القيمة غائبة. ?? قيمة بديلة غير ملي تكون على 0.

تمرين

علاش 0 ?? stock مناسب؟ حيث مخزون 0 قيمة صحيحة. علاش as Product ماشي حماية؟ حيث ما كيبدلش البيانات الخارجية ولا كيتحقق منها.

02 • async و await و try/catch

اتصال قاعدة البيانات والشبكة كيحتاجو وقت. Promise كتمثل نتيجة غادي تجي من بعد.

backend/src/server.ts

backend/src/server.ts

```
 8  async function startServer() {
 9      try {
10          await sequelize.authenticate();
11          console.log('Database connection has been established successfully.');
12          app.listen(PORT, () => {
13              console.log(`Server is running on port ${PORT}`);
14          });
15      }
16      catch (error) {
17          console.error('Unable to connect to the database:', error);
18          process.exitCode = 1;
19      }
20  }
21
22  startServer();
```

سطر بسطر

async كتخلي الدالة ترجع Promise. await sequelize.authenticate كينتظر التأكد من الاتصال بالقاعدة. app.listen كيبدا كيتنفذ ملي السيرفر يبدا يسمع listen ديال callback. غير بعد نجاح الاتصال HTTP استقبال.

إلا وقع خطأ

catch كيلتقط فشل الربط، كيطبع الخطأ وكيحدد exitCode = 1. ماشي نفس هاد الرمز كيخبر النظام أن التشغيل فشل. وكيصلح نسدو finally. فيه المعالجة catch. فيه الخدمة اللي تقدر تفشل try أو connection كيتنفذ بعد النجاح أو الفشل، res.status(500)، حيت هنا مازال كنشغلو العملية.

try / catch / finally

نطفيو loading.

Promise.all

كيشغل عمليات مستقلة مع بعضيتها وكيجمع النتائج. إلا وحدة فشلات كيترفض التجميع؛ ما كيلغيش آلياً الباقي. فعملية شراء مترابطة كنحتاجو transaction، ماشي غير Promise.all.

03 • app.ts: ترتيب استقبال الطلب

هاد الملف كيبني تطبيق Express ويربط كل مجموعة URLs بالـrouter ديالها.

backend/src/app.ts

backend/src/app.ts

```
14  const appserver = express();
15
16  // 1. Security Headers via Helmet
17  appserver.use(helmet());
18
19  // 2. Restrictive CORS
20  appserver.use(cors(corsOptions));
21
22  // 3. Global Rate Limiting
23  appserver.use(globalRateLimiter);
24
25  // 4. Body parsing
26  appserver.use(express.json());
27
28  // API Documentation & Specs (Swagger UI at /api/docs)
```

المراحل

express() كينشئ التطبيق. helmet كيزيد headers أمنية. cors كيتحقق من origin حسب الإعداد. globalRateLimiter كيحد عدد الطلبات. express.json كيحوّل body JSON لـobject فـreq.body.

backend/src/app.ts

```
32  appserver.use("/api/auth", authRouter);
33  appserver.use("/api/products", productsRouter);
34  appserver.use("/api/categories", categoriesRouter);
35  appserver.use("/api/orders", ordersRouter);
36  appserver.use("/api/documents", documentsRouter);
37  appserver.use("/api/agent", agentRouter);
38
39  appserver.get('/health', (req: Request, res: Response) => {
40      res.status(200).json({ message: ' oki Server is healthy' });
41  });
42
```

تركيب URL

الـ/api/auth prefix مع /login داخل authRouter كيعطي /api/auth/login/. هاد التقسيم كيخلي app.ts صغير، وكل router مسؤول على مجال واحد.

الخطأ والصحة

كيرجع جواب ثابت؛ نجاحه بوحدو ما /health. من المسارات السابقة next(err) مركب فالآخر باش يستقبل errorHandler خدامين RAG وMCP وLLM كيثبتش أن.

04 • middleware: حراس قبل الخدمة

Middleware دالة كتدخل بين الطلب والتنفيذ. إما تكمل بـnext، وإما تجاوب وترجع.

backend/src/middlewares/auth.ts
backend/src/middlewares/roles.ts
backend/src/middlewares/tenant.ts

backend/src/middlewares/auth.ts

```
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    res.status(401).json({ error: "Authentication token required" });
    return;
  }

  const token = authHeader.substring(7).trim();

  try {
    const payload = verifyAccessToken(token);
    req.user = payload;
    next();
  } catch (error) {
    res.status(401).json({ error: "Invalid or expired authentication token" });
  }
}

```

شرح المقطع

headers.authorization فيه Bearer ثم التوكن. substring(7) كينحي كلمة Bearer والمسافة. verifyAccessToken كيتحقق من التوقيع والصلاحية. payload كيتحط فـreq.user. next كيمرر للمرحلة اللي من بعد.

علاش return مهم؟

ملي رجعنا 401 خاص الدالة توقف. إلا كملات حتى next، تقدر تنفذ خدمة لمستخدم ما ثبتش هويتو أو ترسل جوج أجوبة.

requireRole

هاد middleware كيتأكد أن الدور داخل اللائحة المسموحة. authenticate كيعرف شكون دخل، وrequireRole كيتحقق واش يقدر يدير العملية. إخفاء زر ADMIN فالواجهة ما كيعوضوش.

resolveStoreTenant و injectTenant

الأول كيستعمل storeId من الهوية المتحقق منها وكيحيد تأثير القيمة المرسلة من العميل. الثاني يسمح بقراية كاتالوغ متجر الديمو بلا دخول، أو يستعمل المتجر من JWT إلا كان موجود وصالح.

05 • Zod: التحقق وقت التشغيل

الطلب الخارجي ما نثقوش فالشكل ديالو. Zod كيرجع بيانات مقبولة أو أخطاء قبل الوصول للخدمة.

backend/src/middlewares/validate.ts
backend/src/schemas/order.ts
backend/src/schemas/auth.ts
backend/src/schemas/product.ts
backend/src/schemas/category.ts

backend/src/middlewares/validate.ts

```
 9      const parseResult = schema.safeParse(req[source]);
10
11      if (!parseResult.success) {
12        res.status(400).json({
13          error: "Validation failed",
14          details: parseResult.error.flatten().fieldErrors,
15        });
16        return;
17      }
18
19      // Replace with typed, sanitized data
20      // Express 5 makes req.query and req.params getter-only properties on the prototype
21      if (source === "body") {
22        req.body = parseResult.data;
23      } else {
```

safeParse

كيعطي success: true ومعاه data، أو success: false ومعاه error. flatten().fieldErrors كينظم أخطاء الحقول. من بعد النجاح req.body كيتعوض بالبيانات اللي خرجات من schema.

backend/src/schemas/order.ts

```
 8  export const createOrderSchema = z.object({
 9    confirmed: z.literal(true),
10    items: z
11      .array(orderItemInputSchema)
12      .min(1, "Order must contain at least one product"),
13    customerName: z.string().trim().optional(),
14    customerPhone: z.string().trim().optional(),
15    shippingAddress: z.string().trim().optional(),
16  });
17
18  export const orderConfirmationSchema = z.object({
```

شرح قواعد الطلب

z.literal(true) كيقبل غير true، ماشي false ولا النص "true". items خاصها عنصر واحد على الأقل. كل عنصر عندو productId و quantity موجب. عدد صحيح من 1 لفوق. optional كيسمح بغياب الحقل.

infer و coerce

z.infer كيشتق type ديال TypeScript من schema. كيحول قيمة نصية بحال "42" لعدد قبل باقي التحقق coerce.number باش ما نكتبووش نفس الوصف مرتين.

05 • Zod: التحقق وقت التشغيل • تابع

تفصيل Express

باش يحط البيانات المتحققة Object.defineProperty كيستعمل params وquery مباشرة. بالنسبة body كيعالج validate فنسخة Express المستعملة هنا.

06 • register: خلق الحساب خطوة بخطوة

الواجهة كترسل الاسم والبريد وكلمة السر، والخادم هو اللي كيحدد المتجر والدور.

backend/src/routes/auth.ts

backend/src/routes/auth.ts

```
74      const passwordHash = await bcrypt.hash(password, 10);
75
76      const user = await User.create({
77        storeId: store.id,
78        name,
79        email,
80        password: passwordHash,
81        role: "CUSTOMER",
82      });
83
84      // Ensure customer record exists for store orders
85      await Customer.findOrCreate({
86        where: { storeId: store.id, email },
87        defaults: {
88          storeId: store.id,
89          name,
```

قبل هاد المقطع

كيشوف واش نفس البريد User.findOne. كيقلب على متجر الديمو Store.findOne. كيتحقق من body registerSchema موجود فنفس المتجر؛ إلا موجود كيرجع 409.

المقطع

جا من storeId. CUSTOMER كيخزن الحساب بالدور User.create. بتكلفة 10 hash كيحسب bcrypt.hash(password,10) المتجر اللي اختارو الخادم. كلمة السر الأصلية ما كتترجعش فالجواب.

Customer

ما خاصوش يتخلط مع User معرف. email وstoreId كيضمن ملف زبون للطلبات مرتبط بـ Customer.findOrCreate هما جوج جداول عندهم دور مختلف. Customer معرف.

نهاية المسار

كيرجع بيانات المستخدم المسموحة والتوكنات. الرد ما res.status(201).json. refresh وaccess كينشئ issueTokenPair ديال كلمة السر hash فيهش.

حد التنفيذ

خطوات إنشاء User وCustomer والتوكنات ماشي مجمعة كلها فمعاملة واحدة هنا. خاص نفهمو هاد الفرق قبل ما نقولو التسجيل كله atomic.

07 • JWT و إصدار زوج التوكنات

Access token بطاقة موقعة للاستعمال مع الطلبات. Refresh token سر عشوائي لتجديد الجلسة.

backend/src/config/jwt.ts
backend/src/routes/auth.ts

backend/src/routes/auth.ts

```
35    const rawRefreshToken = generateRefreshToken();
36    const hashedRefreshToken = hashToken(rawRefreshToken);
37    const expiresAt = new Date(Date.now() + REFRESH_TOKEN_TTL_MS);
38
39    await RefreshToken.create({
40      storeId: user.storeId,
41      userId: user.id,
42      token: hashedRefreshToken,
43      expiresAt,
44    });
45
46    return { accessToken, refreshToken: rawRefreshToken };
47  }
```

raw و hashed

القيم المخزنة، refresh كيتخزن فالقاعدة. إلا تسرب جدول hashedRefreshToken. كيمشي للعميل rawRefreshToken ماشي هي التوكنات الخام اللي كتتقدم للتجديد.

backend/src/config/jwt.ts

```
29  export function generateRefreshToken(): string {
30    return crypto.randomBytes(40).toString("hex");
31  }
32
33  export function hashToken(token: string): string {
34    return crypto.createHash("sha256").update(token).digest("hex");
35  }
```

randomBytes و SHA-256

باش نقدر نقلب SHA-256 كيحسب بصمة hashToken. كيحولها لنص hexكيولد 40 بايت عشوائية، و randomBytes(40) هنا لسر عشوائي قوي SHA-256لكلمات السر، و bcrypt. على توكن بلا تخزينو خام.

Access token

سبعة أيام. التوقيع ما refreshالمدة الافتراضية 15 دقيقة، و .email وroleو storeIdو userId كيوقع signAccessToken ؛. ما نحطوش فيه أسرارJWT كيخبيش محتوى

الإعداد الحالي

معروف ما يصلحش كأساس fallback قوي ومضبوط؛ JWT_SECRET ثابت محلي للسر. فالإنتاج خاص fallback فيه jwt.ts آمن للإنتاج. القيمة نفسها ما محتاجينهاش فهاد الدليل.

08 • login ثم refresh ثم logout

هاد المسارات كيتعاونو باش يبقى المستخدم داخل بلا ما يعاود كلمة السر كل دقيقة.

backend/src/routes/auth.ts

backend/src/routes/auth.ts

```
140     const isMatch = await bcrypt.compare(password, user.password);
141     if (!isMatch) {
142       return res.status(401).json({ error: "Invalid email or password" });
143     }
144
145     const { accessToken, refreshToken } = await issueTokenPair(user);
146
147     return res.status(200).json({
148       message: "Login successful",
149       user: {
```

login

الخادم كيلقى الحساب بنفس المتجر والبريد. bcrypt.compare كيقارن الكلمة المدخلة مع hash. إلا الحساب أو الكلمة غلط، نفس رسالة Invalid email or password كتخرج. إلا صحيح كنصدر زوج توكنات.

backend/src/routes/auth.ts

```
200     // ROTATION: Invalidate the previous refresh token
201     await tokenRecord.destroy();
202
203     // Issue a brand-new token pair
204     const { accessToken, refreshToken } = await issueTokenPair(user);
205
206     return res.status(200).json({
207       message: "Token refreshed successfully",
208       accessToken,
```

refresh

كنحسبو hash ديال التوكن المرسل، كنلقاو السجل، كنشوفو expiresAt والمستخدم. كنحيدو القديم وكنصدرو الجديد. الاستعمال اللاحق للقديم كيترفض ملي السجل تحيد.

logout و me

logout كيمسح سجل refresh المطابق. /me كيرجع الهوية المستخرجة من JWT، ماشي قراءة جديدة كاملة لملف User. لذلك الاسم فالواجهة بعد الاسترجاع كيتعوض مؤقتاً باسم مشتق من البريد.

الحدود

بقفل، لذلك تجربة تسلسلية ناجحة ما transaction هنا ماشي rotation. القديم يقدر يبقى صالح حتى expires. فنفس اللحظة refresh كتثبتش سلامة سباق جوج.

09 • models و migrations و associations

Model كيوصف التعامل مع سجل فالكود. Migration كتبدل بنية قاعدة البيانات. Association كتربط النماذج ببعضياتها.

backend/src/models/Product.ts
backend/src/models/index.ts
backend/src/models/tenantScope.ts

backend/src/models/Product.ts

```
64      price: {
65        type: DataTypes.DECIMAL(10, 2),
66        allowNull: false,
67      },
68      sku: {
69        type: DataTypes.STRING,
70        allowNull: true,
71      },
72      stock: {
73        type: DataTypes.INTEGER,
74        allowNull: false,
75        defaultValue: 0,
76      },
77    },
78    {
```

قراءة Product.init

DataTypes.DECIMAL(10,2) كيوصف الثمن بعشرة أرقام إجمالاً وجوج بعد الفاصلة. stock 0 عدد صحيح وافتراضه 0. allowNull:false كيمنع NULL، ولكن ماشي بوحدو شرط stock >= 0.

Primary و foreign keys

Primary key كيحدد سجل وحيد. Foreign key كيربط عمود بسجل فجدول آخر. وجود productId كـnumber فـinterface ما كينشئش foreign key؛ migration هي اللي كتدير القيد فالقاعدة.

backend/src/models/tenantScope.ts

```
13  export function addStoreScope(model: TenantModel): void {
14    model.addScope(
15      "forStore",
16      (storeId: number) => ({ where: { storeId: validateStoreId(storeId) } }),
17      { override: true }
18    );
19  }
20
21  export function forStore(model: TenantModel, storeId: number): TenantModel {
22    return model.scope({ method: ["forStore", validateStoreId(storeId)] });
23  }
```

forStore

addStoreScope كيسجل scope اسمي كيضيف where.storeId. forStore كيطبقو بعد التحقق أن المعرف موجب وآمن. كل query خاصو scope أو شرط واضح؛ Sequelize ما كيعرفش المتجر الحالي بوحدو.

09 • models و migrations و associations • تابع

Associations

Order.hasMany(OrderItem) يـ كتسمحinclude items. OrderItem.belongsTo(Product) السطر كتسمح بمنتوج.
aliases بحال items وproduct أسماء توافق خاصها include فالمسارات.

10 • GET products: البحث والتقسيم

هاد المسار كيحول الفلاتر اللي فالواجهة لشروط استعلام محددة.

backend/src/routes/products.ts

backend/src/routes/products.ts

```
35        const page = query.page || 1;
36        const limit = query.limit || 20;
37        const offset = (page - 1) * limit;
38
39        const whereClause: any = {};
40
41        // Keyword search on name or description (case-insensitive)
42        const searchTerm = query.q || query.search;
43        if (searchTerm) {
44          whereClause[Op.or] = [
45            { name: { [Op.iLike]: `%${searchTerm}%` } },
46            { description: { [Op.iLike]: `%${searchTerm}%` } },
47          ];
48        }
49
50        // Category filter
```

حساب الصفحات

page=1 وlimit=20 كيعطيو offset=0. page=2 كتعطي offset=20. offset هو عدد السجلات اللي غادي نتجاوزو. schema كتحد القيم قبل الحساب.

الفلاتر

Op.or كيعني الاسم أو الوصف. Op.iLike بحث نصي ما كيفرقش الحروف الكبيرة والصغيرة. Op.gte هو أكبر أو يساوي، وOp.lte أصغر أو يساوي. categoryId كيزيد شرط الصنف.

backend/src/routes/products.ts

```
67        const scopedProduct = forStore(Product, storeId);
68        const { rows: products, count: total } = await scopedProduct.findAndCountAll({
69          where: whereClause,
70          include: [
71            {
72              association: "category",
73              attributes: ["id", "name"],
74              required: false,
75            },
76          ],
77          order: [["id", "ASC"]],
78          limit,
79          offset,
```

findAndCountAll

rows و الصفحة الحالية هي count هو العدد المطابق. include category كيجيب صنف المنتوج. required:false كيسمح بمنتوج بلا صنف. Math.ceil(total/limit) كيعطي عدد الصفحات.

10 • GET products: البحث والتقسيم • تابع

مساران آخران

القراءة الحالية ما كتحجزش السلعة؛ inStock. كيجيب الثمن والمخزون الحاليين مع /:id/stock/كيجيب التفاصيل، و /:id الحجز الفعلي كيتقرر وقت معاملة الطلب.

11 • تعديل المنتجات والأصناف

الإدارة كتحتاج التحقق من الدور والمتجر والمدخلات، حتى إلا الواجهة عطلات الأزرار لغير المسؤول.

backend/src/routes/products.ts
backend/src/routes/categories.ts

backend/src/routes/products.ts

```
175  productsRouter.post(
176    "/",
177    authenticate,
178    injectTenant,
179    requireRole("ADMIN"),
180    validate(createProductSchema, "body"),
181    async (req: Request, res: Response, next: NextFunction) => {
182      try {
183        const storeId = req.storeId as number;
184        const { name, description, imageUrl, price, stock, sku, categoryId } = req.body;
185
186        // Verify category belongs to this store if provided
```

ترتيب الحماية

داخل category من بعد الخدمة كتقلب على validate. ثم requireRole("ADMIN") ثم injectTenant ثم authenticate إذا تعطى قبل إنشاء المنتوج SKU نفس المتجر، وكتفحص.

backend/src/routes/products.ts

```
270        await product.update({
271          ...(req.body.name !== undefined ? { name: req.body.name } : {}),
272          ...(req.body.description !== undefined
273            ? { description: req.body.description }
274            : {}),
275          ...(req.body.imageUrl !== undefined ? { imageUrl: req.body.imageUrl } : {}),
276          ...(req.body.price !== undefined ? { price: req.body.price } : {}),
277          ...(req.body.stock !== undefined ? { stock: req.body.stock } : {}),
278          ...(req.body.sku !== undefined ? { sku: req.body.sku } : {}),
279          ...(req.body.categoryId !== undefined
280            ? { categoryId: req.body.categoryId }
281            : {}),
```

التعديل الجزئي

... هو spread كيدخل خصائص object داخل object آخر. الشرط !== undefined كيفرق بين حقل ما ترسلش وبين قيمة بحال 0 أو null. بهاذ الطريقة stock=0 يتطبق فعلاً.

الأصناف

categories.ts فيه list/detail/create/update/delete. الاسم كيتفحص داخل المتجر قبل الإنشاء أو التعديل.
product.categoryId رابط لصنف؛ ماشي النص ديال اسم الصنف مخزن فكل منتوج.

الحذف

كيطلب حذف السجل. إلا عندو علاقات بقيود فالقاعدة، النتيجة كتتبع القيود. تنبيه الحذف فالواجهة كيساعد destroy المستخدم ولكن ما كيغيرش قواعد قاعدة البيانات.

12 • إنشاء طلب من REST

POST /api/orders كيحتاج confirmed:true وكيحسب الثمن من القاعدة.

backend/src/routes/orders.ts
backend/src/schemas/order.ts

backend/src/routes/orders.ts

```
60        const result = await sequelize.transaction(async (t) => {
61          let total = 0;
62          const orderItemsData: Array<{
63            storeId: number;
64            customerId: number;
65            productId: number;
66            quantity: number;
67            unitPrice: number;
68            name: string;
69          }> = [];
70
71          // Check stock and lock product rows
72          for (const item of items) {
73            const product = (await Product.findOne({
74              where: { id: item.productId, storeId },
```

بداية المعاملة

sequelize.transaction(async t => ...) كيدير commit إلا callback نجح، و rollback تابع query إلا رمى خطأ. كل وتحديث الهاتف اللي قبلها خارج هاد المعاملة transaction:t. resolveCustomer للمعاملة خاصو.

backend/src/routes/orders.ts

```
73            const product = (await Product.findOne({
74              where: { id: item.productId, storeId },
75              transaction: t,
76              lock: t.LOCK.UPDATE, // Exclusive lock: guarantees concurrency safety
77            })) as Product | null;
78
79            if (!product) {
80              const err = new Error(`Product #${item.productId} not found in this store`);
81              (err as any).statusCode = 404;
82              throw err;
```

القفل

كيقفل السجل فالمعاملة. من بعد كيتراجع المخزون؛ إذا ما lock: t.LOCK.UPDATE فيه where وstoreId وproductId كافيش throw كيوقف العملية ويرجع أثرها.

الحساب والتخزين

كيخلق طلب Order.create. كينقص المخزون decrement. كيتجمع بالكمية product.price. total كيتقرا من unitPrice وقت الشراء. كيحفظ السطور ب OrderItem.bulkCreate unitPrice PENDING.

12 • إنشاء طلب من REST • تابع

بعد commit

ما يرجعش البيع لور. الجواب 201 فيه الطلب والسطور n8n ؛ فشلvoidكيتنادى بعد المعاملة بـ notifyOrderCreated.
ولكن ماشي إثبات تشفيري أن شخص ضغط زر فعلاً، API شرط confirmed:true.

13 • قراية الطلب وإلغاؤه

نفس storeId ما كافيش؛ الزبون خاص يشوف غير طلباتو.

backend/src/routes/orders.ts

backend/src/routes/orders.ts

```
282         if (!customer || order.customerId !== customer.id) {
283           return res.status(404).json({ error: "Order not found" });
284         }
285       }
286
287       return res.json({ order });
```

GET /orders

إذا CUSTOMER، كنلقاو Customer بالبريد والمتجر وكنفلتروا الطلبات بـADMIN .customerId يقدر يشوف طلبات متجره. include كيرجع سطور الطلب والمنتجات المرتبطة.

GET /orders/:id

أولاً طلب من نفس المتجر، ثم ملكية الزبون. الطلب ديال شخص آخر كيرجع 404 باش ما نكشفوش وجوده بسهولة. الدور موثوق لأنه من JWT متحقق منه.

backend/src/routes/orders.ts

```
342         if (order.status === "SHIPPED" || order.status === "DELIVERED") {
343           const err = new Error(
344             `Cannot cancel an order that has already been ${order.status.toLowerCase()}`
345           );
346           (err as any).statusCode = 400;
347           throw err;
348         }
349
350         // Retrieve items for this order
```

الإلغاء العادي

كيقفل الطلب، كيتحقق من الملكية ومن الحالة، كيجيب السطور ويرجع confirmed:true. POST /:id/cancel كيطلب المخزون بـincrement ثم يبدل الحالة CANCELLED داخل المعاملة.

PATCH ديال ADMIN

وكيعيد المخزون عند الانتقال للإلغاء. ما كايناش آلة انتقالات كاملة تمنع جميع القفزات enum كيقبل حالة من /:id/status بين الحالات؛ خاص ما نصفوش كل انتقال بأنه محمي بقواعد تجارة كاملة.

14 • RAG: upload والـtransaction

الملف المرفوع كيتحول لنص، ثم مقاطع، ثم embeddings، ثم سجلات قابلة للبحث.

backend/src/routes/documents.ts

backend/src/routes/documents.ts

```
13  const upload = multer({
14    storage: multer.memoryStorage(),
15    limits: { fileSize: 2 * 1024 * 1024 },
16    fileFilter: (_req, file, cb) => {
17      const allowed = ["text/plain", "text/markdown"];
18      if (allowed.includes(file.mimetype) || file.originalname.match(/\.(md|txt)$/i)) {
19        cb(null, true);
20      } else {
21        cb(new Error("Only .md and .txt files are accepted"));
22      }
23    },
24  });
25
```

Multer

memoryStorage الصيغ المقبولة. بايت 1024*1024*2 الحد. فالذاكرة الملف كيحط md وtxt حسب MIME أو الامتداد. upload.single("file") كيتوقع اسم الحقل file فـFormData.

الدخول

لجميع مسارات الوثائق. التعليق اللي كيقول ADMIN وinjectTenant وauthenticate كيركب documentsRouter.use متاح لكل مستخدم ما كيبدلش هاد الحماية العامة search.

backend/src/routes/documents.ts

```
153       const textChunks = chunkText(rawText);
154
155       if (textChunks.length === 0) {
156         await transaction.rollback();
157         return res.status(422).json({ error: "Document has no usable text content" });
158       }
159
160       // 3. Generate embeddings (batched, single API call for ≤100 chunks)
161       const embeddings = await embedChunks(textChunks);
162
163       // 4. Bulk-insert all chunks with their embeddings
164       await Chunk.bulkCreate(
165         textChunks.map((content, i) => ({
166           storeId,
```

الخطوات

Document كيتخلق processing داخل transaction. chunkText كيقسم rawText. 422 كيرجع ملف بلا نص مفيد. embedChunks ثم كينادي المزود bulkCreate ديالو والمتجه كل مقطع محتوى كيحفظ.

14 • RAG: upload والـtransaction • تابع

النجاح والفشل

processing. برسالة آمنة failed ثم سجل rollback يعني المقاطع تحفضو. إلا وقع فشل كنديرو commitو completed أخرى أثناء الانتظار connection داخل نفس المعاملة ماشي بالضرورة حالة مرئية للائحة من.

15 • chunkText: تتبع المؤشر

هاد الدالة كتخدم بالنص فقط، بلا database وبلا LLM.

backend/src/services/rag.ts

backend/src/services/rag.ts

```
27    const normalised = text.replace(/\r\n/g, "\n").trim();
28    if (normalised.length === 0) return [];
29
30    const chunks: string[] = [];
31    let start = 0;
32
33    while (start < normalised.length) {
34      let end = Math.min(start + CHUNK_SIZE, normalised.length);
35
36      // If we haven't reached the end, try to break at a natural boundary
37      if (end < normalised.length) {
38        // Prefer paragraph break
39        const paraBreak = normalised.lastIndexOf("\n\n", end);
40        if (paraBreak > start + CHUNK_OVERLAP) {
```

normalised و start و end

كنوحدو نهايات السطور وكنحيدو الفراغ من الطرفين. start هو بداية المقطع. end هو الأصغر بين start+800 ونهاية النص. while كتكمل مادام start ما وصلش للنهاية.

حدود طبيعية

قبل القطع كنجربو نلقاو نهاية فقرة، سطر أو مسافة. lastIndexOf كيقلب لور من end. الهدف ما نقطعوش الكلمة وسطها إلا ما لقيناش حد مناسب.

backend/src/services/rag.ts

```
58      const chunk = normalised.slice(start, end).trim();
59      if (chunk.length > 0) {
60        chunks.push(chunk);
61      }
62
63      // Move forward with overlap
64      start = Math.max(start + 1, end - CHUNK_OVERLAP);
65    }
66
67    return chunks;
```

slice و overlap

كتاخذ الجزء بين start وend. push فاللائحة. كتزيدو end-100 كيخلي تداخل مع المقطع السابق.

Math.max(start+1,...) كيضمن المؤشر يتقدم وما يبقاش loop لا نهائية.

ملاحظة فعلية فالكود

ملي end توصل لنهاية النص، ما كاينش break؛ start كيرجع لور بمقدار overlap ثم كيتقدم. هذا يقدر يولد مقاطع ذيلية متكررة بزاف. الهدف ~800/100 واضح، ولكن التنفيذ الحالي خاصو مراجعة نهاية الحلقة.

16 • embedding ثم SQL similarity

البحث كيقارن تمثيل السؤال العددي بتمثيلات المقاطع داخل متجر واحد.

backend/src/services/rag.ts
backend/sql/001_init_pgvector.sql

backend/src/services/rag.ts

```
76  export async function embedText(text: string): Promise<number[]> {
77    const response = await openai.embeddings.create({
78      model: EMBEDDING_MODEL,
79      input: text,
80    });
81    return response.data[0].embedding;
82  }
83
```

embedChunks و embedText

الأولى كتبعث نص واحد وترجع embedding من الجواب. الثانية كتقسم لائحة المقاطع لدفعات 100 وتجمع المتجهات. النموذج الافتراضي text-embedding-3-small، والعمود مضبوط على 1536 بُعد.

backend/src/services/rag.ts

```
158      SELECT
159        c.id                                    AS "chunkId",
160        c."documentId"                          AS "documentId",
161        d.name                                  AS "source",
162        c.content                               AS "content",
163        1 - (c.embedding <=> :vector::vector)   AS "score"
164      FROM chunks c
165      JOIN documents d ON d.id = c."documentId"
166      WHERE c."storeId" = :storeId
167        AND c.embedding IS NOT NULL
168        AND 1 - (c.embedding <=> :vector::vector) >= :threshold
169      ORDER BY c.embedding <=> :vector::vector
170      LIMIT :topK
171      `,
```

شرح SQL

JOIN هي <=>. كيربط المقطع بالوثيقة باش نجيبو اسم المصدر. cosine distance. 1-distance كتستعمل كscore. WHERE كيحدد storeId ومتجه غير فارغ و scoreفوق العتبة. ORDER BY وLIMITكيقدم الأقرب وكيحد العدد.

replacements

القيم كتتعطى منفصلة عبر أسماء vector وstoreId وthreshold وtopK. ما كنركبوش السؤال الخام داخل نص SQL. هادي كتقلل خطر حقن SQL فهاد الاستعلام.

العثور ماشي صحة مؤكدة

0.75 عتبة قرب دلالي ماشي نسبة احتمال صحة. HNSW كيسرع البحث التقريبي. خاص تقييم أسئلة حقيقية باش تختار العتبة، وكتراجع الوثيقة نفسها واش صحيحة.

17 • حذف وإعادة فهرسة الوثائق

وجود Document ماشي هو وجود معرفة مفهرسة وقابلة للاسترجاع.

backend/src/routes/documents.ts
backend/src/seed.ts

اللائحة GET

كتجمع العدد Map. لكل وثيقة chunksCreated كيحسب groupو COUNT مع Chunk.findAll كيجيب findAll metadata. فالجواب بلا قراءة النصوص كاملة document مع كل.

reingest

كيحيد المقاطع القديمة، كيقسم وكيولد transaction، كيبدا PUT /:id/reingest كيتحقق من الوثيقة داخل المتجر، إلا فشل كيرجع rollback. commit. الجديدة ثم embeddings

delete

كيجيب الوثيقة بالمتجر ثم كيحذف chunks بـstoreIdو documentIdوالوثيقة داخل transaction. من بعد الحذف ما خاصش هاد المقاطع تبقى مصدر للاسترجاع.

backend/src/seed.ts

```
174        await Chunk.findOrCreate({
175          where: { storeId: store.id, documentId: document.id },
176          defaults: {
177            storeId: store.id,
178            documentId: document.id,
179            content: definition.content,
180            embedding: null,
181          },
182          transaction,
183        });
184      }
185    });
```

تفصيل seed

المقطع هنا embedding:null، والوثيقة فretrieveContext كيستبعد seed pending. الفارغ. خاص ingestion أو reingestion حقيقية باش وثائق الديمو تدخل البحث المتجهي.

ترتيب route

خاص ترتيب أكثر دقة؛ agent هاد endpoint كأنه id. يقدر ياخذ search فهاد router. Express معلنة قبل /search قبل /:id كيستعمل retrieveContext مباشرة، ما كيمرش من endpoint search.

18 • llm.ts: الاتصال بالمزود

هاد الملف كيجمع إعدادات الاتصال وتعليمات المساعد وآلية قراءة streaming من المزود.

backend/src/services/llm.ts

backend/src/services/llm.ts

```
31  export function getLlmClient(): OpenAI {
32    return new OpenAI({
33      apiKey: getLlmApiKey(),
34      baseURL: getLlmBaseUrl(),
35      timeout: 30_000, // 30 s hard timeout per request
36      maxRetries: 2, // retry twice on transient 5xx / network errors
37    });
38  }
39
40  export const LLM_MODEL =
```

إعدادات المزود

getLlmClient كينشئ SDK ب timeout 30 ثانية. كيقراو أسماء البيئة بالترتيب المحدد فـ getLlmApiKey وgetLlmBaseUrl. maxRetries=2 وهادي مهلة للطلب، ماشي ضمان أن الدور كامل يسالي فـ30 ثانية.

buildSystemPrompt

كيوصف دور ShopMate واللغة والمصادر والرفض والأدوات. النص كيتضاف لـmessages. التحقق من الملكية والصلاحيات خاصو يبقى حتى فالدوال البرمجية.

backend/src/services/llm.ts

```
144     const createStream = (includeUsage: boolean) => client.chat.completions.create(
145       {
146         model,
147         messages,
148         tools: tools && tools.length > 0 ? tools : undefined,
149         tool_choice: tools && tools.length > 0 ? "auto" : undefined,
150         stream: true,
151         ...(includeUsage ? { stream_options: { include_usage: true } } : {}),
152         max_tokens: MAX_TOKENS_PER_TURN,
153         temperature: 0.3, // low temp = factual, consistent shop assistant
154       },
155       { signal }
156     );
157     let stream;
158     try {
```

parameters

stream:true يخلي النموذج يقترحها؛ tool_choice:auto وصف القدرات؛ tools الحوار؛ messages اسم النموذج؛ model تقلل التنوع وماشي ضمان صحة temperature:0.3 حد خرج الطلب؛ max_tokens يطلب جواب متدرج؛.

التوافق

الكود كيجرب include_usage. إلا المزود رفض هاد الخيار، كيعاود بلا بيه. usage الغائبة خاصها تبقى غير متوفرة، ما نفهموهاش كاستهلاك صفر.

19 • streamChat: جمع أجزاء الجواب

المزود يقدر يرجع النص أو اسم الأداة وarguments مقطعين على عدة chunks.

backend/src/services/llm.ts

backend/src/services/llm.ts

```
170     for await (const chunk of stream as AsyncIterable<ChatCompletionChunk>) {
171       if (chunk.usage) {
172         callbacks.onUsage?.({
173           promptTokens: chunk.usage.prompt_tokens,
174           completionTokens: chunk.usage.completion_tokens,
175           totalTokens: chunk.usage.total_tokens,
176         });
177       }
178       const delta = chunk.choices[0]?.delta;
179       if (!delta) continue;
180
181       // Text token
182       if (delta.content) {
183         fullText += delta.content;
184         callbacks.onToken(delta.content);
185       }
186
187       // Tool call chunks
```

for await

كتقرا stream جزء بجزء. delta هو التغيير الجديد. fullText كيجمع النص. onToken callback كيبلغ المستعمل ديال هاد الدالة؛ فـagent.ts المستعمل كيخزن النص حتى يدوز moderation.

Tool calls

كل tool call عندو index وaccumulatedToolCalls. id كيجمع name وarguments اللي جاو على دفعات. JSON ما خاصوش يتنفذ قبل ما يكتمل؛ onToolCalls كيتنادى بعد نهاية الجمع.

ترتيب callbacks

ثم يقرر واش ينادي messagesيقدر يضيف نتائج الأدوات ل agent بهذا. onDone قبل awaited كيكون onToolCalls النموذج مرة أخرى.

Abort و error

هادشي مختلف عن .onErrorكيتعطى لطلب المزود. إذا تبطل، الدالة كتخرج بهدوء. خطأ آخر كيمشي لـ AbortSignal جواب رفض عادي من قواعد المتجر.

chat()

الدالة الأخيرة كتطلب completion بلا streaming وترجع content أو نص فارغ. ما تفترضش أن كل export مستعمل فالمسار الرئيسي؛ agent عندو حتى loop JSON خاص به.

20 • agent.ts: بداية الدور

هاد route هو المنظم: هوية → محادثة → حمايات → مصادر → نموذج وأدوات → جواب وسجل.

backend/src/routes/agent.ts

backend/src/routes/agent.ts

```
49  const chatMessageSchema = z.object({
50    message: z.string().trim().min(1).max(2000),
51    conversationId: z.coerce.number().int().positive().optional(),
52  });
53
```

المدخلات

مركبين على injectTenant وauthenticate. اختياري موجب conversationId. trim بين 1 و2000 حرف بعد message كامل، و router على agentRateLimiter كامل /chat.

backend/src/routes/agent.ts

```
174       if (conversationId) {
175         conversation = (await forStore(Conversation, storeId).findOne({
176           where: { id: conversationId, userId: user.userId },
177         })) as Conversation | null;
178       }
179       if (!conversation) {
180         conversation = await Conversation.create({
181           storeId,
182           userId: user.userId,
183           title: userText.slice(0, 40),
184           status: "active",
185         });
186       }
187
188       // ── 2. Persist user Message & bump conversation timestamp ──────────────
```

المحادثة

البحث كيجمع id وuserId داخل scope المتجر. إذا ما لقاهاش كيخلق جديدة بعنوان أول 40 حرف. معرفة conversationId ديال شخص آخر ما كتسمحش بتحميل محادثتو.

الحفظ والحماية

الرسالة checkProhibitedIntent. ثم checkPromptInjection كيحفظ سؤال المستخدم. بعدها Message.create المشبوهة كتتعوض فالتاريخ بنص محجوب باش ما يعاودش النموذج يتعرض ليها كحوار عادي.

المسار القصير

الرفض كيتسجل ويرجع بلا استدعاء LLM. التأكيد على pending action حتى هو مسار مباشر، ولكن بعد فحص الرسالة الممنوعة باش ما تؤكدش عملية بالخطأ.

21 • agent.ts: السياق والتاريخ

السؤال الحالي كيحتاج سياق، ولكن كل سياق خاصنا نعرفو المصدر والثقة ديالو.

backend/src/routes/agent.ts

backend/src/routes/agent.ts

```
328       const retrievedChunks = await retrieveContext(activeUserText, storeId);
329       const ragChunks = retrievedChunks.filter((chunk) => {
330         const suspicious = checkPromptInjection(`${chunk.source}\n${chunk.content}`).isSuspicious;
331         return !suspicious;
332       });
333       ragSources = retrievedChunks.map((chunk) => ({
334         name: chunk.source,
335         score: Number(chunk.score),
336         omitted: !ragChunks.some((safeChunk) => safeChunk.chunkId === chunk.chunkId),
337       }));
338
339       // ── 4. Build initial message history ──────────────────────────────
340       const history = await Message.findAll({
341         where: { conversationId: conversation.id },
```

RAG filtering

retrieveContext كيرجع المقاطع القريبة. filter إذا كيستبعد المقطع فيه نمط مشبوه content أو source. ragSources باش يبقى الأثر omitted كيسجل حتى واش المقطع.

backend/src/routes/agent.ts

```
340       const history = await Message.findAll({
341         where: { conversationId: conversation.id },
342         order: [["createdAt", "ASC"]],
343         limit: 10,
344         attributes: ["role", "content"],
345       });
346
347       const initialMessages: LLMMessage[] = [
```

تركيب messages

كيتغلف RAG فيه المقاطع المنقاة، ثم السؤال الحالي. سياق context أولاً، ثم جزء من التاريخ، ثم System prompt مع تعليمات اعتباره بيانات context بعلامات.

ملاحظة على history

الاستعلام ASC مع limit:10 كيجيب أول عشرة، ماشي آخر عشرة. history.slice(0,-1) مفترض يحيد السؤال اللي تزاد، ولكن بعد طول المحادثة هاد الافتراض ما يبقاش صحيح. خاص تشرح الحد الحالي بوضوح.

سؤال للفهم

واش النموذج كيذكر كل محادثات الناس؟ لا. الخادم هو اللي كيختار messages من المحادثة المسموحة ويرسلها فكل طلب. الذاكرة هنا مخزنة فالقاعدة ومركبة بالكود.

22 • loop ديال الأدوات

اقتراح الأداة من النموذج كيرجع للخادم؛ الخادم ينفذ ويرجع النتيجة للنموذج.

backend/src/routes/agent.ts
backend/src/services/agentTools.ts

backend/src/routes/agent.ts

```
454                   const validation = validateToolCall(toolName, tc.function.arguments, toolContext);
455                   const result = await executeToolCall(toolName, tc.function.arguments, toolContext);
456                   toolsCalled.push({
457                     name: toolName,
458                     args: validation.success ? sanitizeForLogging(validation.args) : null,
459                     validationPassed: validation.success,
460                     ...(validation.success ? {} : { validationError: validation.error }),
461                   });
462                   allToolsUsed.push(toolName);
463                   toolCallCount++;
464
465                   // Emit tool_result event — treated as data by client, not instructions
```

ثلاث خطوات

validateToolCall كيتحقق. executeToolCall كيعيد التحقق وينفذ. toolsCalled كيسجل args وحالة validation المتحققة. toolCallCount كيزيد حتى إلا كانت النتيجة خطأ، باش المحاولات ما تكونش بلا حد.

ديال الأدوات messages

الرسالة assistant كتحتوي tool_calls. من بعد كنضيفو role:tool مع tool_call_id باش المزود يعرف نتيجة أي استدعاء. content منقى بـsanitizeContextData.

الحد 3

كيسمح مادام العدد أقل من 3. الحد كيتطبق على الاستدعاءات المنفذة، ماشي على عدد الكلمات canCallAnotherTool أو عدد رسائل المستخدم.

الحد الحالي

ملي كيتستنفد الحد أثناء loop JSON، tool calls يقدر يبقى آخر عنصر فيه role:tool ويخرج finalResponse فارغ. كذلك دفعة فيها أدوات أكثر من السقف تحتاج إكمال بروتوكول النتائج بعناية. هادشي نقطة مراجعة، ماشي وعد بجواب كامل فكل حالة.

العلاقة بالثمن

الأداة كتجيب رقم حقيقي من DB، ولكن صياغة LLM النهائية ممكن تبقى تخطئ. مراجعة المخرجات الحالية ما كتقارنش كل رقم بالناتج بشكل شامل.

23 • allowlist و Zod و dispatcher

هاد الطبقة كتحدد شنو يقدر المساعد يدير وبأي مدخلات.

backend/src/services/agentTools.ts
backend/src/schemas/agentTools.ts

backend/src/services/agentTools.ts

```
317    delete parsedArgs.storeId;
318    delete parsedArgs.userId;
319    delete parsedArgs.customerId;
320
321    // 4. Validate against Zod schema
322    const schema = TOOL_SCHEMAS[name];
323    const parseResult = schema.safeParse(parsedArgs);
324
325    if (!parseResult.success) {
326      const formattedErrors = parseResult.error.issues
327        .map((issue) => `${issue.path.join(".") || "arguments"}: ${issue.message}`)
328        .join(", ");
329
```

قبل المقطع

الاسم خاصو يكون داخل ALLOWED_TOOLS. rawArguments إذا نص، JSON.parse كيقراه. malformed JSON كيرجع خطأ. خاص حتى الحالات JSON بحال null تتراجع قبل التعامل معها كobject.

المقطع

كنحيدو storeId وuserId وcustomerId من اقتراح النموذج. schema.safeParse كيتحقق من باقي args. context الحقيقي كيبقى من الخادم. حذف identity من args ما يعنيش أن جميع النتائج مصرح بها؛ كل أداة خاصها فلترة.

dispatcher

tool. ديال كود كتب النموذج eval كيربط الاسم بالدالة الفعلية. ما كاينش validation.toolName على switch كيطبقها وقت التنفيذ Zodاللي كتتبعت للمزود كتشرح المدخلات، و definitions.

أدوات القراية

مخزون وثمن؛ checkStockTool تفاصيل؛ getProductDetailsTool لائحة محدودة وفلاتر؛ searchProductsTool كيتحقق من getShipmentStatusTool. آخر الطلبات بسقف20 listMyOrdersTool طلب مملوك؛ getOrderStatusTool الملكية ثم MCP.

تنبيه التأكيد

يقدر ينادي dispatcherمن النموذج، و confirmed:boolean كتقبل createOrder/cancelOrder ديال schema موجود. هذا ما كيعطيش وحدو ضمان أن التأكيد جا من المستخدم؛ مع pending إذا true executeConfirmedAction خاص ربط الإثبات بمسار موثوق.

24 • recap و pending confirmation

initiateCreateOrderRecap كتقرا المعطيات وتحضر ملخص، وتخزن نية العملية مؤقتاً.

backend/src/services/orderConfirmation.ts

backend/src/services/orderConfirmation.ts

```
175    pendingConfirmations.set(conversationId, {
176      token,
177      conversationId,
178      storeId,
179      userId: context.userId,
180      action: "createOrder",
181      args,
182      recapText: recap,
183      totalAmount: calculatedTotal,
184      createdAt: Date.now(),
185      executed: false,
186    });
187
188    return {
```

القيم

token معرف عشوائي للملخص؛ conversationId رابط المحادثة؛ storeIdو userId الهوية؛ args العناصر؛ totalAmount المجموع؛ createdAt وقت البداية؛ executed واش تنفذات.

قبل الحفظ

الدالة كتقرا كل product داخل المتجر، كتتأكد من stock وكتحسب subtotal ثم calculatedTotal. كتجهز recap فيه المبلغ وCOD والعنوان والهاتف.

فالذاكرة Map

pendingConfirmations كتموت مع restart وما كتتشركش بين نسختين API. getPendingConfirmation وhasPendingConfirmation كينفذو مهلة15 دقيقة. ما نفسروش هاد التخزين على أنه سجل دائم.

affirmative

isAffirmativeConfirmation كتصغر الحروف وتحيد بعض العلامات وكتقلب على oui/ok/wakha وغيرها. مطابقة كلمة Oui، داخل جملة تقدر تغلط مع النفي. زر الواجهة كيرسل ولكن النص الحر خاص تعامل أقوى.

رفض الملخص

الواجهة كترسل Non, annuler وكتبدل حالة البطاقة. الكود اللي كيجهز pending ما فيهش مسار واضح لمسحها بهاد النص. ما نساويوش تغيير لون البطاقة مع إلغاء pending فالخادم.

25 • executeConfirmedAction: الشراء الفعلي

التنفيذ كيقفل المخزون ويكتب الطلب، مع تنسيق مؤقت ضد تكرار نفس العملية.

backend/src/services/orderConfirmation.ts

backend/src/services/orderConfirmation.ts

```
266    const activeExecution = confirmationExecutions.get(conversationId);
267    if (activeExecution) return activeExecution;
268
269    const execution = performConfirmedAction(conversationId, context);
270    confirmationExecutions.set(conversationId, execution);
271    try {
272      return await execution;
273    } finally {
274      confirmationExecutions.delete(conversationId);
275    }
276  }
277
278  async function performConfirmedAction(
```

Promise مشتركة

إلا نفس conversation عندها تنفيذ جاري، كنرجعو نفس Promise. finally كيمسحها من confirmationExecutions. pending.executedResult كيسمح بإرجاع نتيجة سابقة بدل كتابة جديدة.

performConfirmedAction

كتجيب pending ثم تختار createOrder أو cancelOrder. مسار الإنشاء كيعيد قراية المنتجات بقفل ومعاملة، كينقص المخزون، كيخلق Order بحالة CONFIRMED وOrderItems.

فرق REST والشات

جوج مسارات كينفذو منطق متشابه فملفين؛ لهذا CONFIRMED. والشات المؤكد كيبدأه PENDING، كيبدأ الطلب REST خاص مقارنة السلوك عند أي تعديل.

حدود خاص ما نخبيوهاش

بشكل مستقل. ومسار الإلغاء ما pendingوالهوية المخزنة ف TTL ما كتعاودش تحقق performConfirmedAction وقت التنفيذ. ضمانات المسار الأعلى ما كتغنيش عن حماية REST ديال SHIPPED/DELIVERED كيعاودش نفس منع الدالة نفسها.

بيانات العنوان

العنوان كيبان فالrecap ولكن نموذج Order الحالي ما فيهش shippingAddress. فـREST حتى الحقل المقبول فـschema ما كيتحفظش فـOrder.create. اختيار الحجم فالواجهة حتى هو ما كيمشيش فالطلب. هادو حدود بيانات، ماشي مجرد تنسيق.

26 • guardrails.ts بالتفصيل

هاد الملف فيه قواعد نصية للرفض وتنقية السياق ومراجعة الجواب.

backend/src/services/guardrails.ts

backend/src/services/guardrails.ts

```
72  export function checkPromptInjection(input: string): InjectionCheckResult {
73    const matchedPattern = detectInjectionPattern(input);
74
75    // Strip raw system tag delimiters that attempt to fake conversation boundaries
76    const sanitizedText = input
77      .replace(/<\/?system>/gi, "")
78      .replace(/<\/?instructions>/gi, "")
79      .replace(/<\|im_start\|>|<\|im_end\|>/gi, "")
80      .replace(/\[\/?INST\]/gi, "")
81      .replace(/<<SYS>>|<\/SYS>>/gi, "");
82
83    return {
84      isSuspicious: Boolean(matchedPattern),
85      detectedPattern: matchedPattern?.source,
```

RegExp

كيشوف واش النص كيوافق نمط regex.test. كيرجع أول نمط مطابق find. كينحي علامات تحاول تنتحل system أو instructions. الأنماط ما كتفهمش كل معاني اللغة.

sanitizeContextData

إذا النص فيه injection معروفة، كتبدلو برسالة محتوى محجوب. إلا ما بانش نمط، كتنحي بعض tags. هادي كتستعمل للوثائق ونتائج الأدوات.

checkProhibitedIntent

كتجرب مجموعات discount ثم refund ثم بيانات الآخرين ثم medical/legal ثم off-topic. كترجع shouldRefuse وcategory وfalse. refusalMessage معناها الأنماط ما شداتش الطلب، ماشي إثبات أن كل محتوى آمن.

moderateAgentOutput

كتبحث على تسريب تعليمات/أسرار ووعود تخفيض معروفة، وترجع passed والنص البديل. ما كتتحققش من كل ادعاء طبي ولا من كل رقم مقارنة بDB.

الرسالة والإشعار

العبارة ما خاصهاش تعتبر دليل توصل. n8n best-effort كتقول تم الإحالة، بينما remise ديال STANDARD_REFUSALS المسؤول؛ خاص فصل عرض الإحالة عن نجاح الإرسال.

27 • SSE: النص النهائي كيخرج بعد المراجعة

الاتصال واحد، والخادم كيكتب فيه أحداث JSON مفصولة بسطرين.

backend/src/routes/agent.ts
mobile/src/api/agentChat.ts

backend/src/routes/agent.ts

```
166     const safeSend = (data: object): void => {
167       if (isSSE && clientConnected && !res.writableEnded) {
168         res.write(`data: ${JSON.stringify(data)}\n\n`);
169       }
170     };
171
```

wire format

data: ثم JSON ثم سطرين. type كيحدد شنو وقع: start للمحادثة، source للمصادر، tool_call/tool_result للأدوات، token للنص، done للنهاية، error للفشل.

backend/src/routes/agent.ts

```
495                 // Enforce the RAG abstention in code as well as in the prompt.
496                 // Live DB tools remain valid sources for stock, price, and order actions.
497                 if (ragChunks.length === 0 && allToolsUsed.length === 0) {
498                   fullResponse = "Je n'ai pas cette information.";
499                   refusalCategory = "no_rag_context";
500                 }
501
502                 if (!clientConnected) return; // client disconnected mid-stream
503
504                 safeSend({ type: "token", token: fullResponse });
505
506                 const assistantMsg = await Message.create({
```

منع اختلاق جواب بلا مصدر

إذا ما كاين لا ragChunks لا allToolsUsed، النص النهائي كيتعوض بجملة عدم المعرفة. استعمال أداة فاشلة كيبقى محسوب فـallToolsUsed؛ لذلك هاد الشرط ماشي إثبات أن كل أداة أعطات سند صالح.

أولاً moderation

كيخرج النص النهائي safeSend، ثم moderation. من بعد كيدوز fullResponse كيجمع provider ديال onToken. تقدر توصل أثناء العمل، والنص ما كيتعرضش حرف بحرف فور توليده telemetry.

فصل الاتصال

النسخة الحالية كتسمع res.on("close") وكتفحص res.writableEnded! قبل abort. هادي أحسن دلالة على انقطاع الجواب من إغلاق request وحده؛ التحقق العملي من كل مزود باقي مستقل.

28 • AgentLog: شنو كيتسجل؟

السجل مخصص لتتبع دور المساعد، والرسائل مخصصة لتاريخ الحوار.

backend/src/routes/agent.ts
backend/src/middlewares/sanitizeLogs.ts
backend/src/models/AgentLog.ts

backend/src/routes/agent.ts

```
 130        await AgentLog.create({
 131          storeId,
 132          agentName: logText(process.env.LLM_MODEL ?? LLM_MODEL),
 133          action: entry.action,
 134          status: entry.status,
 135          conversationId: conversation?.id ?? null,
 136          messageId: traceMessageId,
 137          input: logText(userText).slice(0, 4000),
 138          output: entry.output === undefined ? null : logText(entry.output).slice(0, 4000),
 139          errorMessage: entry.errorMessage === undefined ? null : logText(entry.errorMessage).slice(0,
2000),
 140          toolsCalled: sanitizeForLogging(toolsCalled) as Array<Record<string, unknown>>,
 141          ragSources: sanitizeForLogging(ragSources) as Array<Record<string, unknown>>,
 142          tokenUsage,
 143          latencyMs: Date.now() - turnStartedAt,
 144          refusalCategory,
 145        });
 146      };
 147      const addTokenUsage = (usage: { promptTokens: number; completionTokens: number; totalTokens: number
}): void => {
```

مفاتيح التتبع

conversationId وmessageId كيربطو السجل بالمحادثة والجواب. input/output النصوص؛ toolsCalled فيه args المتحققة؛ ragSources المصادر؛ latencyMs فرق Date.now؛ tokenUsage مجموع الأرقام اللي عطا المزود.

التنقية

sanitizeForLogging كيمشي داخل arrays وobjects، فالنص كيحجب أسماء الحقول الحساسة وأنماط أسرار access tokens. مقاييس عددية مسموحة، ماشي promptTokens/completionTokens.

الطول والغياب

هنا أثر منظم null. therefore full trace تقدر تكون usage. عند2000 error مقطوعين عند4000 حرف و input/output محدود، ماشي نسخة غير محدودة من كل شيء.

GET /logs

endpoint فهاد customerId. فلاتر page/limit/date/status/disposition/conversation. مع ADMIN store scope. عبر User ID كيتفسر كـ Conversation.userId، ماشي Customer.id ديال الطلبات.

حدود التغطية

طلبات مرفوضة فـmiddleware قبل route، بحال401 أو429، ما كتوصلش لـrecordTurn. وتنقية AgentLog ما كتطبقش آلياً على كل console.error فالمشروع ولا على كل Message.

29 • MCP client والخادم المنفصل

getShipmentStatusTool كيتأكد من صاحب الطلب قبل التواصل مع خدمة التتبع.

backend/src/services/mcpShipmentClient.ts
backend/src/mcp/shipmentServer.ts

backend/src/services/mcpShipmentClient.ts

```
44      await client.connect(transport);
45      const result = await client.callTool({
46        name: "getShipmentStatus",
47        arguments: { orderId, orderStatus },
48      });
49      if (result.isError) throw new Error("MCP shipment tool returned an error");
50
51      const text = result.content.find((item) => item.type === "text")?.text;
52      if (!text) throw new Error("MCP shipment tool returned no status data");
53      const parsed = shipmentResultSchema.safeParse(JSON.parse(text));
54      if (!parsed.success) throw new Error("MCP shipment tool returned invalid status data");
55      return parsed.data;
56    } finally {
```

الخطوات

كيعرف transport وطريقة URL وHTTP. client.connect كيبدا التواصل. callTool و كيرسل الاسم وorderId وorderStatus. قبل إرجاع البيانات Zod وJSON.parse كيتقلب فيه على نص، ومن بعد result.content.

finally

client.close كيتنادى حتى إلا وقع خطأ. fetchWithTimeout كيصنع AbortController لكل fetch، وكيحيد ومؤقت5 ثوان لكل المؤقت والمستمع من بعد. المدة الإجمالية لعدة طلبات بروتوكول ماشي بالضرورة5 ثوان فقط.

الخادم

shipmentServer كيسجل الأداة بسكيما مدخلات. statusByOrder كيحول الحالة الداخلية لوصف نقل محاكى trackingNumber مشتق من orderId، والموعد التقديري محسوب من الوقت الحالي؛ ماشي معلومة من ناقل حقيقي.

Node HTTP

الخادم كيجمع body، كيبني Web Request، كيمررو للhandler، وكيحول Web Response لجواب Node. SIGINT/SIGTERM كينادیو shutdown باش يسد الموارد.

الأمان

خادم MCP المعروض ما عندوش مصادقة مستقلة فهاد الملف. التحقق التجاري موجود فAPI قبل الاتصال، وهذا ما كيمنعش وحده الاتصال المباشر بخادم MCP إذا كان مكشوفاً.

30 • n8n والإشعار بعد commit

إشعار المسؤول عملية خارجية اختيارية، منفصلة على نجاح معاملة الشراء.

backend/src/services/n8nNotifications.ts
backend/n8n/shopmate-manager-notifications.json

backend/src/services/n8nNotifications.ts

```
54      const response = await fetch(webhookUrl, {
55        method: "POST",
56        headers,
57        body: JSON.stringify(payload),
58        signal: AbortSignal.timeout(WEBHOOK_TIMEOUT_MS),
59      });
60
61      if (!response.ok) {
62        console.warn(`n8n manager notification failed with HTTP ${response.status}.`);
63        return false;
64      }
65      return true;
66    } catch {
67      // Do not log the payload: it can contain customer contact information.
```

sendToN8n

كيقرا URL من البيئة؛ إلا غائب يرجع false. headers فيها JSON وممكن body. X-ShopMate-Webhook-Secret فيه الحدث والتفاصيل. AbortSignal.timeout كيوقف الطلب الطويل.

نوعا الحدث

notifyOrderCreated كيزيد event:order.created وcreatedAt. notifyManagerRequest كيزيد manager.request المكنة payload. TypeScript union كيوصف أشكال للخصم أو الاسترجاع.

best effort

response.ok نجاح كيعطي HTTP. فشل الشبكة أو رد غير ناجح كيرجع false بلا payload. ما وكيكتب تحذير مختصر دائمة وإعادة إرسال مضمونة فهاد الملف queue كايناش.

workflow JSON

ملف n8n هو وصف قابل للاستيراد. خاص إعداد credentials والقناة وتفعيل webhook فخدمة n8n. مجرد وجود الملف فالريبو ما كيعنيش أنه خدام.

31 • React: state و props و hooks

Component دالة كتنتج واجهة. State بيانات كتبدل العرض، وprops مدخلات جاية من component آخر.

mobile/src/components/Button.tsx
mobile/app/(auth)/login.tsx

mobile/app/(auth)/login.tsx

```
30    const [email, setEmail] = useState("");
31    const [password, setPassword] = useState("");
32    const [clientError, setClientError] = useState<string | null>(null);
33
34    const validate = (): boolean => {
35      setClientError(null);
36      clearError();
37
38      if (!email.trim()) {
39        setClientError("L'adresse email est requise.");
40        return false;
41      }
```

useState

email هي القيمة الحالية. setEmail كتطلب تحديثها وإعادة رسم الواجهة. TextInput.value و onChangeTextكيعرضها كيربط الكتابة بـsetEmail. هادشي يسمى controlled input.

useEffect و useCallback و useMemo

useEffect كيشغل أثر بحال تحميل API بعد render. dependencies كتحكم إمتى يعاود. useCallback كيحافظ على مرجع دالة حسب dependencies. useMemo كيحافظ على نتيجة حساب، ماشي على بيانات دائمة.

mobile/src/components/Button.tsx

```
38    const isDisabled = disabled || loading;
39
40    return (
41      <Pressable
42        onPress={onPress}
43        disabled={isDisabled}
44        style={({ pressed }) => [
45          styles.base,
46          styles[variant],
47          styles[`size_${size}`],
48          pressed && styles.pressed,
49          isDisabled && styles.disabled,
50          style,
51        ]}
52      >
```

والشرط props

Button كيتلقى title وonPress وloading وغيرها. loading أو disabled كيعطلو الضغط. styles array كيجمع style ديال المتصفح native. HTML وصف الواجهة، ماشي return داخل JSX. الأساسي والنوع والحجم والحالة

31 • React: state و props و hooks • تابع

useRef

كيحتافظ بقيمة بين renders بلا ما يسبب render عند تغييرها. مناسب لمرجع stream اللي نحتاجو نوقفوه أو FlatList باش نديرو scroll.

32 • navigation و استرجاع الدخول

Expo Router كيربط بنية الملفات بالشاشات، وRootLayout كيتابع حالة الجلسة.

mobile/app/_layout.tsx
mobile/app/(tabs)/_layout.tsx
mobile/app/(auth)/_layout.tsx

mobile/app/_layout.tsx

```
47      const inAuthGroup = segments[0] === "(auth)";
48
49      if (!isAuthenticated && !inAuthGroup) {
50        // Unauthenticated user trying to access protected screens -> redirect to login
51        router.replace("/(auth)/login");
52      } else if (isAuthenticated && inAuthGroup) {
53        // Authenticated user trying to access login/register -> redirect to tabs
54        router.replace("/(tabs)");
55      }
56    }, [isAuthenticated, isLoading, segments, router]);
57
58    if (isLoading) {
```

الرحلة

كيعرف واش useSegments من بعد splash. صحيح كيبان isLoading كيتنادى عند البداية. مادام initializeAuth المستخدم فمجموعة auth. router.replace كيدخلو login أو tabs حسب حالة الجلسة.

Stack و Tabs

Stack كينظم شاشة فوق أخرى بحال تفاصيل منتوج. Tabs كيخلي التنقل بين Commandesو Boutiqueو Assistant وProfil. (auth)و (tabs)و مجموعات تنظيمية فالأسماء.

[id]

product/[id].tsx route ؛ ديناميكية /product/42 كتعطي id داخل 42 نصي useLocalSearchParams. خاص تحويله والتحقق قبل طلب API.

الإدارة

href:null كيخبي صفحات الإدارة من شريط tabs. كل شاشة كتراجع role، والخادم حتى هو كيراجع ADMIN. إخفاء route ماشي حاجز أمان كافي.

33 • authStore و tokenStorage

Zustand كيجمع حالة الجلسة والدوال اللي كتغيرها فبلاصة واحدة.

mobile/src/stores/authStore.ts
mobile/src/auth/tokenStorage.ts

mobile/src/stores/authStore.ts

```
127         const { user, accessToken, refreshToken } = response.data;
128
129         await saveTokens({ accessToken, refreshToken });
130
131         set({
132           user,
133           accessToken,
134           refreshToken,
135           isAuthenticated: true,
136           isLoading: false,
137           error: null,
138         });
139
140         return true;
141       } catch (err: any) {
```

login و register

الدالة كتدير loading، كترسل API، كتخزن التوكنات، ثم set كيحدث user وisAuthenticated. الفشل كيرجع false ويخزن error باش الشاشة تعرضه.

initializeAuth

كتقرا tokens من التخزين. إذا غايبين الجلسة خارجية. إذا موجودين كتطلب /me؛ Axios يقدر يجدد access. كتعاود تقرا التوكنات من التخزين باش ما تستعملش القيمة القديمة بعد rotation.

التخزين

قابل localStorage مستوى الحماية مختلف؛ localStorage. كتستعمل web نسخة SecureStore. كيستعمل native ديال المنتجات DB ديال نفس الأصل. ما نخلطوش هاد التخزين مع JavaScript للقراية من.

logout

كيحاول يبطل refresh فالخادم ثم clearTokens وset كيرجعو الحالة فارغة. إذا الشبكة فشلات، الخروج المحلي كيوقع، ولكن الإبطال فالخادم ما تأكدش.

set و get

فالشاشة كيتابع الجزء اللي محتاجاه useAuthStore(selector). كيقرا الحالة الحالية داخل الدوال get، كيبدل الحالة set بحال role أو isLoading.

34 • Axios interceptors و refreshPromise

باش ما نكتبوش Authorization ومعالجة401 فكل شاشة، client كيجمعهم.

mobile/src/api/client.ts
mobile/src/config/api.ts

mobile/src/api/client.ts

```
28  apiClient.interceptors.request.use(
29    async (config: InternalAxiosRequestConfig) => {
30      const { accessToken } = await getTokens();
31      if (accessToken && config.headers) {
32        config.headers.Authorization = `Bearer ${accessToken}`;
33      }
34      return config;
35    },
36    (error) => Promise.reject(error)
37  );
38
```

قبل الإرسال

getTokens كيجيب access الحالي. interceptor كيضيف Bearer للheaders ثم يرجع config. apiClient عندو baseURL وJSON timeout10 ثوان للطلبات العادية.

بعد401

response interceptor كيتأكد من error.config ويمنع إعادة login/refresh أو request عندها _retry. ثم كيستعمل refreshPromise نفس مشتركة، باش عدة طلبات ما يجددوش فآن واحد refresh.

mobile/src/api/client.ts

```
110         const newAccessToken = await refreshPromise;
111
112         // Update failed request's Authorization header and retry
113         if (originalRequest.headers) {
114           originalRequest.headers.Authorization = `Bearer ${newAccessToken}`;
115         }
116
117         return apiClient(originalRequest);
118       } catch (refreshError) {
119         // Refresh failed or revoked: log out cleanly
```

إعادة الطلب

بعد التجديد، التوكن الجديد كيتخزن وcallbacks كتخبر authStore. Authorization ديال الطلب الفاشل كيتبدل ثم apiClient(originalRequest) كيعيدو مرة واحدة.

العنوان

كيتوجه10.0.2.2، والتليفون الحقيقي يحتاج defaultHost. Android emulator كيتقدم على EXPO_PUBLIC_API_URL منشور HTTPS. localhost فالتليفون هو التليفون نفسه. عنوان LAN أو مناسب

35 • الكاتالوغ فالموبايل

الشاشة كتعرض وتجمع الاختيارات، والـstore كيرسل الفلاتر وكيحتافظ بالنتائج.

mobile/src/stores/catalogStore.ts
mobile/app/(tabs)/index.tsx
mobile/app/product/[id].tsx

mobile/src/stores/catalogStore.ts

```
124        const response = await apiClient.get<{
125          products: Product[];
126          pagination: PaginationData;
127        }>("/api/products", { params });
128
129        const products = response.data.products || [];
130        const pagination = response.data.pagination;
131
132        set({
133          products,
134          hasMore: pagination ? pagination.page < pagination.totalPages : false,
135          page: 1,
136          status: "success",
137          error: null,
138        });
```

fetchProducts

كتحضر params من search/category/minPrice/maxPrice، كتطلب الصفحة الأولى وكتعوض products. hasMore كيتحسب من page وstatus. totalPages كيوضح loading أو success أو error.

loadMore

كيمنع الطلب إذا ما بقاش صفحات أو تحميل جاري. nextPage=page+1، ثم products:[...products,...newProducts] كتجمع القديم والجديد. دير بالك من نتائج طلب قديم بعد تبديل filters؛ الكود ما عندوش إلغاء شامل لهاد السباقات.

FlatList

كيطلب المزيد؛ onEndReached معرف ثابت؛ keyExtractor كيفاش نرسم عنصر؛ renderItem اللائحة؛ data RefreshControl لإعادة التحميل. الحالة الفارغة مختلفة على حالة الخطأ.

تفاصيل المنتوج

كيظهر handleOrderCOD. إذا الصورة غائبة أو فشلات fallback عندها stock. Image كتجيب المنتج ثم fetchDetails الحالي bodyالحجم المختار ما كيتبعتش ف. confirmed:true وitems وبعد التأكيد كيرسل Alert.

36 • streamAgentChat: فك أحداث SSE

الشات كيستعمل XMLHttpRequest مباشرة باش يقرا جواب كيتزاد تدريجياً.

mobile/src/api/agentChat.ts

mobile/src/api/agentChat.ts

```
80    const processBuffer = () => {
81      const newText = xhr.responseText.slice(lastIndex);
82      lastIndex = xhr.responseText.length;
83      buffer += newText;
84
85      const parts = buffer.split("\n\n");
86      // Keep incomplete trailing event chunk in buffer
87      buffer = parts.pop() ?? "";
88
89      for (const part of parts) {
90        const trimmed = part.trim();
91        if (!trimmed) continue;
92
93        const lines = trimmed.split("\n");
94        for (const line of lines) {
```

buffer و lastIndex

كيحتافظ بحدث ناقص، و buffer. كتاخذ غير الجديد slice(lastIndex) فيه جميع البيانات الواصلة responseText. بسطرين كيعزل الأحداث الكاملة parts.pop كيرجع الباقي غير المكتمل.

dispatch

سطر كيبدأ data: كيتحول من JSON لـswitch event. كينادي callback المناسب. onStart يحفظ conversationId، onSources يعرض المصادر، onToolResult يجهز البطاقات، onToken يزيد النص.

حالات الاتصال

ثم isAborted كيعلم abort. session expirée أثناء التحميل، و4 عند النهاية. 429 رسالة انتظار، و401 readyState3 xhr.abort. onerror كيعالج فشل الشبكة.

فرق مهم عن Axios

هاد XHR ما كيمرش من interceptor ديال apiClient. 401 فالشات ما عندهاش نفس التجديد التلقائي الموجود فالطلبات العادية. fetchConversationHistory وfetchSingleConversation كيمرو عبر Axios.

37 • chat.tsx: النص والبطاقات

الشاشة كتدير state ديال المحادثة، بينما الخادم هو اللي كينفذ الخدمات.

mobile/app/(tabs)/chat.tsx

mobile/app/(tabs)/chat.tsx

```
261      setMessages((prev) => [...prev, userMessage, assistantMessage]);
262      setInput("");
263      setIsSending(true);
264      setAgentStatus("ShopMate AI réfléchit...");
265
266      setTimeout(() => {
267        flatListRef.current?.scrollToEnd({ animated: true });
268      }, 100);
269
270      try {
```

sendMessage

كتحيد الفراغ، كتمنع إرسال فارغ أو مزدوج أثناء isSending، كتزيد رسالة المستخدم وplaceholder فارغ للمساعد. من بعد streamAgentChat كيعطي callbacks لتحديث نفس placeholder بالـid.

تحديث immutable

يعرف React أو نص جديد. هادشي كيخلي {...m, sources}كيرجع لائحة جديدة. العنصر المطلوب كيتبدل بـ prev.map التغيير بلا تعديل عشوائي لنفس object.

البطاقات

searchProducts result.products كيتحول لProductCardData. createOrder requiresConfirmation كتخلق orderRecap. success مع orderId كيخلق createdOrder. onDone كيطفي isStreaming وisSending.

التأكيد والتاريخ

التاريخ DB. حالة البطاقة كتتبدل قبل جواب الخادم، إذن اللون ماشي إثبات نجاح Oui. كيبعث handleConfirmOrder كيرجع الرسائل النصية، وبعض البطاقة كيتعاد بناؤها باستخراج رقم من النص.

اختلاف شكل البيانات

فرع getProductDetails فالواجهة كيتوقع result.product، بينما الأداة الحالية كترجع خصائص المنتوج مباشرة. هذا يفسر علاش بطاقة التفاصيل تقدر ما تظهرش رغم وجود جواب الأداة.

38 • orders و profile والشاشات الإضافية

كل شاشة كتربط state وعناصر العرض بدالة أو API واضحة.

mobile/app/(tabs)/orders.tsx
mobile/app/(tabs)/profile.tsx
mobile/app/+not-found.tsx
mobile/app/+html.tsx
mobile/app/modal.tsx

mobile/app/(tabs)/orders.tsx

```
119               await apiClient.post(`/api/orders/${orderId}/cancel`, {
120                 confirmed: true,
121               });
122               fetchOrders();
123             } catch {
124               Alert.alert("Erreur", "Impossible d'annuler la commande. Réessayez.");
```

OrdersScreen

كيختار اللون والنص getStatusBadge. كيظهر التحميل orders. onRefresh ويعوض /api/orders كيطلب fetchOrders وgetStatusStepIndex كيختار موضع التقدم. الإلغاء كيحتاج Alert ثم confirmed:true ثم إعادة تحميل.

الخطوات المرئية

شريط الحالة فيه En cours، ولكن OrderStatus فالخادم ما فيهش نفس الحالة كقيمة مستقلة. خط تقدم بصري ماشي دائماً model كامل ديال حالات النقل.

ProfileScreen

كيقرا user من store، كيعرض الاسم والبريد والدور، وكيظهر روابط الإدارة للـADMIN. الصورة والموقع الظاهرين فيهم قيم ديمو؛ ماشي تحديد موقع مباشر ولا صورة رفعها كل مستخدم.

الملفات الإضافية

شاشة إضافية. وجود ملف قابل للتوجيه ما كيثبتش web. modal قالب +html. صفحة لمسار غير موجود +not-found أنه جزء ضروري من رحلة الشراء الحالية.

39 • إدارة الكاتالوغ فالموبايل

الفورموليز كيحتافظ بالنصوص، وكيحول الأرقام قبل الإرسال.

mobile/app/(tabs)/admin-catalogue.tsx

mobile/app/(tabs)/admin-catalogue.tsx

```
  67    const saveProduct = async () => {
  68      const price = Number(productDraft.price);
  69      const stock = Number(productDraft.stock);
  70      if (!productDraft.name.trim()) return setError("Le nom du produit est obligatoire.");
  71      if (!Number.isFinite(price) || price <= 0) return setError("Le prix doit être un nombre supérieur à
zéro.");
  72      if (!Number.isInteger(stock) || stock < 0) return setError("Le stock doit être un entier positif ou
zéro.");
  73      setSaving(true);
  74      setError("");
  75      try {
  76        const payload = {
  77          name: productDraft.name.trim(), description: productDraft.description.trim() || null,
  78          price, stock, sku: productDraft.sku.trim() || null, categoryId: productDraft.categoryId,
  79        };
  80        if (editingProduct) await apiClient.put(`/api/products/${editingProduct}`, payload);
  81        else await apiClient.post("/api/products", payload);
```

ProductDraft

price وstock نصوص أثناء الكتابة. Number كيحولهم. Number.isFinite كيرفض NaN وInfinity، وNumber.isInteger كيطلب مخزون صحيح. name.trim كيمنع اسم فراغ.

إنشاء أو تعديل

وكتعاود draft للإنشاء. من بعد النجاح كتفرغ ID بلا POST، للتعديل ID مع PUT. إلا كنعدلو ID فيه editingProduct كيعطل الزر أثناء العملية. saving loadCatalogue.

التصنيفات

useMemo كيبني Map من categoryId للاسم. saveCategory نفس منطق POST/PUT. deleteProduct/deleteCategory كيطلبو Alert ومن بعد DELETE وإعادة تحميل.

حدود الشاشة

رغم أن imageUrl الحالي ما فيهش ProductDraft. إدارة كاملة pagination بلا limit100 كيطلب loadCatalogue API كيدعموه. عرض الصور فالزبون ما كيعنيش أن المسؤول عندو رفع/تعديل صور من هاد الفورموليز وmodel.

40 • إدارة الوثائق والسجلات

كل شاشة كتراجع ADMIN فالواجهة وكتتواصل مع endpoints محمية فالخادم.

mobile/app/(tabs)/admin-documents.tsx
mobile/app/(tabs)/admin-agent-logs.tsx

mobile/app/(tabs)/admin-documents.tsx

```
  81        const form = new FormData();
  82        form.append("file", {
  83          uri: file.uri,
  84          name: file.name,
  85          type: file.mimeType || (file.name.toLowerCase().endsWith(".md") ? "text/markdown" :
"text/plain"),
  86        } as unknown as Blob);
  87        form.append("name", file.name.replace(/\.(md|txt)$/i, ""));
  88
  89        const response = await apiClient.post<{
  90          message: string;
  91          document: { id: number; name: string; status: string; chunksCreated: number };
  92        }>("/api/documents", form, { headers: { "Content-Type": "multipart/form-data" } });
```

رفع الوثيقة

كيعطي native رفع name. وfile فيه FormData كيرجع الملف المختار. كنراجعو الامتداد والحجم، ثم DocumentPicker وكيعاود يحمل اللائحة chunksCreated نجاح الرد كيعرض uri/name/type.

المعالجة

كيعرض uploading ورسالة إنشاء spinner embeddings. ماشي نسبة تقدم حقيقية؛ هادشي حالة محلية عامة. timeout Axios10 ثوان يقدر يكون قصير لعملية embedding كبيرة.

فلاتر السجلات

كيرسل loadLogs. filters والتواريخ ثم يبدل ID كيتحقق من applyFilters المطبقة. filters منفصلين عن draft fields وappend، params كيضيف صفحة أخرى.

التفاصيل

كيعرض السؤال والجواب والأدوات والمصادر والتكلفة والمدة TurnDetail. /api/agent/logs/:id كيطالب showDetail ما خاصهاش تتفسر كأن الخدمة مجانية فtokens null.

41 • theme والمكونات والتصميم

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

```
mobile/src/theme/tokens.ts
mobile/src/components/Button.tsx
mobile/src/components/Input.tsx
mobile/src/components/Card.tsx
mobile/src/components/BrandHeader.tsx
mobile/src/components/Loader.tsx
mobile/src/components/EmptyState.tsx
mobile/src/components/ErrorState.tsx
```

tokens

colorTokens ألوان، spacingTokens مسافات، radiusTokens تدوير، typographyTokens كتابة. تعديل قيمة مركزية كيأثر على المكونات اللي كتستعملها؛ ماشي على كل قيمة مكتوبة يدوياً.

Button و Input

Button كيجمع الشكل والتعطيل والتحميل و onPress. Input كيجمع الحقل والعنوان والخطأ حسب props. props كتخلي نفس component يتستعمل بأشكال متعددة.

Card و BrandHeader

Card كيوفر سطحاً ومسافات وحدوداً. BrandHeader كيوحد رأس الشاشات. children كيدخل المحتوى داخل component عام بدل نسخ نفس هيكل العرض.

ثلاث حالات

Loader انتظار، EmptyState بيانات فارغة، ErrorState فشل مع إمكانية retry. الفراغ ماشي خطأ: لائحة طلبات جديدة ممكن تكون فارغة بشكل طبيعي.

StyleSheet

flex:1 كيعطي مساحة متاحة. flexDirection:row ترتيب أفقي. padding فراغ داخلي، margin خارجي، gap بين العناصر. colors وsizes ما عندهمش تأثير على authorization أو transaction.

stitch_shopmate

المجلد مرجع ماكيت وتصميم. كود React Native هو التنفيذ اللي كيظهر للمستخدم. الشاشة اللي فالصورة ما كتثبتش أنها مربوطة بـAPI.

42 • Dockerfile و docker-entrypoint

الصورة كتجمع التطبيق، وCompose كيربطه بقاعدة البيانات.

backend/Dockerfile
backend/docker-entrypoint.sh
docker-compose.yml
backend/docker-compose.yml

backend/Dockerfile

```
  2  FROM node:22-alpine AS build
  3  WORKDIR /app
  4  COPY package*.json ./
  5  RUN npm ci
  6  COPY tsconfig.json ./
  7  COPY src ./src
  8  RUN npx tsc
  9
```

build و dependencies

npm ci كيستعمل lockfile مضبوط باش التثبيت يكون مرحلة production-dependencies كتثبت غير dependencies اللازمة للتشغيل. npx tsc كيخرج JavaScript فـdist.

runtime

كتنسخ dist وnode_modules والمigrations وUSER shopmate. كيشغل بصلاحيات أقل. EXPOSE توثيق للمنفذ؛ ports فـCompose هي اللي كتربط منفذ الجهاز بالحاوية.

backend/docker-entrypoint.sh

```
  4  echo "Running database migrations..."
  5  ./node_modules/.bin/sequelize-cli db:migrate \
  6    --env production \
  7    --config src/config/sequelize-cli.js \
  8    --migrations-path migrations
  9
 10  echo "Ensuring pgvector index..."
 11  node dist/scripts/applyPgvectorSql.js
 12
 13  if [ -n "${SEED_ADMIN_PASSWORD:-}" ]; then
 14    echo "Running idempotent demo seed..."
 15    node dist/seed.js
 16  else
 17    echo "SEED_ADMIN_PASSWORD is not set; skipping optional demo seed."
 18  fi
 19
```

الترتيب

set -eu كيوقف السكريبت عند أخطاء وأسماء متغيرات غير موجودة حسب الاستعمال. migrations أولاً، pgvector SQL من بعد، seed ثم اختياري، exec node كيخلي عملية Node تستقبل إشارات الإيقاف مباشرة.

42 • Dockerfile و docker-entrypoint • تابع

Compose

db كتستعمل pgvector:pg16 وvolume. API كتستنى DB healthy. DB_HOST=db داخل الشبكة. health ديال API كينادي /health. ملفات env المكونة للحاوية وتوسيع متغيرات Compose خاصها قيم متوافقة.

43 • seed و SQL و migrations

هنا كنجهزو البيانات والبنية باش API تلقى شنو تخدم عليه.

backend/src/seed.ts
backend/src/scripts/applyPgvectorSql.ts
backend/sql/001_init_pgvector.sql

seed()

كيطلب SEED_ADMIN_PASSWORD بطول أدنى12، كيحسب hash وكيخلق أو يحدث admin، كيضمن متجر وأصناف ومنتجات حسب SKU. إعادة التشغيل ما كتخلقش نفس السجل دائماً، ولكن كتقدر تحدث admin وتزيد صور ناقصة.

الوثائق الأولية

حقيقية باش تولي قابلة index خاص .embeddings نصوص ديمو عامة، كتتحفظ بمقاطع بلا knowledgeDocuments للإجابة من وثائق مفهرسة agent بوحدو جهز seed للبحث. ما تقولش.

backend/src/scripts/applyPgvectorSql.ts

```
11      const sqlPath = path.resolve(__dirname, "../../sql/001_init_pgvector.sql");
12      const sql = await readFile(sqlPath, "utf8");
13      await sequelize.query(sql);
14      console.log("pgvector extension and HNSW index are ready.");
15    } finally {
16      await sequelize.close();
17    }
18  }
```

سكريبت SQL

catchكيسد الاتصال، و finally .كينفذو sequelize.query .من مسار محسوب بالنسبة للكود المترجم SQL كيقرا readFile عند الفشل exitCode كيحدد.

up و down

كل migration عندها up للتطبيق وdown للتراجع. أسماء التواريخ كتحدد الترتيب. create-users كتخلق الجدول، align-model-constraints كتبدل قيود، add-role كتضيف الدور وكتحيد قيد متجر واحد/مستخدم واحد القديم.

قبل الحفظ

تعديل Model بوحدو ما كيغيرش قاعدة موجودة تلقائياً فهاد التشغيل. خاص migration مناسبة، وإلا تقدر تشوف column does not exist وقت API كتطلب عمود جديد.

44 • package.json و CI و scripts

npm run كيشغل الأمر المسجل تحت scripts، وماشي جميع أسماء scripts اختبارات.

backend/package.json
mobile/package.json
.github/workflows/backend-tests.yml

Backend

dev يشغل tsx watch للسيرفر. mcp وmcp:dev يشغلو خدمة التتبع. db:migrate يشغل Sequelize CLI بالمسار ثم يشغل مجموعة محددة من ملفات الاختبار TypeScript يبني test. يشغل بيانات البداية seed. والإعداد.

Mobile

start كيبدأ Expo، android وios وweb كيطلبو فتح المنصة المناسبة. هاد الأوامر ماشي بالضرورة إنتاج APK. app.json فيه قواعد الأنواع والمسارات tsconfig، وplugins فيه اسم التطبيق والأيقونة و.

.github/workflows/backend-tests.yml

```
13  jobs:
14    test:
15      runs-on: ubuntu-latest
16      defaults:
17        run:
18          working-directory: backend
19      steps:
20        - uses: actions/checkout@v4
21        - uses: actions/setup-node@v4
22          with:
23            node-version: 22
24            cache: npm
25            cache-dependency-path: backend/package-lock.json
26        - run: npm ci
27        - run: npm test
```

CI

GitHub Actions كيخدم على push وأpull_request حسب paths. checkout كيجيب الكود، setup-node كيجهز Node22، npm ci وثبت npm test. هاد workflow يختبر حقيقي ولا هاتف ما فيهش نشر تلقائي.

شنو ما درناش هنا

فجلسة إعداد الدليل ما شغلناش اختبارات التطبيق. راجعنا محتوى السكريبتات باش نفهمو نطاقها. نجاح build ديال الوثيقة ماشي نجاح التطبيق.

45 • قراءة test: arrange / act / assert

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

backend/src/scripts/test-agent-cost-limits.ts
backend/src/scripts/test-auth-tenant.ts
backend/src/scripts/test-order-confirmation.ts
backend/src/scripts/test-agent-tools.ts
backend/src/scripts/test-read-tools.ts
backend/src/scripts/test-guardrails.ts
backend/src/scripts/test-n8n-notifications.ts
backend/src/scripts/test-llm.ts

backend/src/scripts/test-agent-cost-limits.ts

```
  36      for (let i = 0; i < 21; i += 1) {
  37        const response = await fetch(url, {
  38          method: "POST",
  39          headers: { "content-type": "application/json", connection: "close" },
  40          body: JSON.stringify({ message: `message ${i + 1}` }),
  41        });
  42        statuses.push(response.status);
  43        lastBody = await response.text();
  44      }
  45
  46      assert("first 20 messages are accepted", statuses.slice(0, 20).every((status) => status === 200));
  47      assert("the 21st message receives HTTP 429", statuses[20] === 429);
  48      assert("429 response contains a mobile-friendly message", lastBody.includes("max 20 messages /
minute"));
  49    } finally {
```

cost limits

السكريبت كيدير خادم Express مؤقت ويعطي user اختباري، ويرسل21 طلب. كيتأكد الأول20 عندهم200 والواحد21 عندو429. فحص ثابت1024 ما كيقيسش فاتورة provider كاملة.

auth-tenant

كيستعمل HTTP حقيقي محلي مع بدائل لجزء من models: دخول، refresh rotation، رفض القديم، logout وعزل منتوج/طلب. هذا مفيد للـroute contract وماشي برهان لكل قيود SQL.

confirmation

كيختبر كلمات التأكيد وschema REST. كيجرب اتصال DB؛ إذا متوفر يدير حالات DB، وإذا غائب يمشي mocks. خاص تقرا أي فرع خدم فالنتائج قبل ما تقول integration حقيقية.

باقي scripts

لخادم n8n للأنماط والرفض والمخرجات؛ guardrails لقراءات الأدوات؛ read-tools للتحقق والـallowlist؛ agent-tools webhook؛ اختباري llm اتصال بالمزود. test:read-tools وtest:llm ما داخلينش فالأمر الحالي npm test.

46 • كيفاش تتبع bug وتجاوب اللجنة

شرح الكود كيبان ملي تقدر تربط نتيجة فالشاشة بسببها فالخادم.

مثال: الشات رجع401

بدا من mobile/src/api/agentChat.ts: XHR كيحمل access من التخزين. شوف واش صالح؛ هاد المسار ما كيديرش refresh ديال Axios. من بعد authenticate فالخادم كيقرر401. ما تبدأش بتغيير RAG.

مثال: وثيقة موجودة وما كتستعملش

شوف status وchunks، ومن بعد embedding واش NULL، ومن بعد storeId والعتبة. seed بوحدو ما كيعمرش المتجهات. وثيقة مفهرسة ممكن تبقى ما تجيبش السؤال بسبب similarity.

مثال: طلب نجح وما وصلش إشعار

الـtransaction commit أولاً، n8n من بعد وبشكل best-effort. شوف إعداد webhook وحالة HTTP وخدمة n8n؛ فشل الإشعار ما كيعنيش نقص المخزون ترجع.

عرض فدقيقتين

ورّي الشاشة اللي كتجمع الطلب، دالة query، schema، route، API client ب transaction، storeId إذا كتابة، ثم response وتحديث state. هادي رحلة كاملة مفهومة أكثر من سرد أسماء التقنيات.

أجوبة صادقة

قول: هاد الجزء قرأتو وها مسارو. هاد الحالة جربتها وها الدليل. هاد الحد لقيتو ومازال خاصو إصلاح. استعمال AI فالبناء ما يمنعش التعلم؛ المهم تقدر تفسر السلوك والحدود.

47 • قاموس ملفات المشروع

هاد الفهرس كيغطي ملفات المصدر ديال backend وmobile والـmigrations وإعدادات التشغيل المختارة. الصور وlockfiles والمكتبات المولدة ماشي scripts ديال منطق التطبيق.

.github/workflows/backend-tests.yml

اختبارات scripts وماشي جميع أسماء scripts، كيشغل الأمر المسجل تحت npm run.

backend/docker-compose.yml

الصورة كتجمع التطبيق، وCompose كيربطه بقاعدة البيانات.

backend/docker-entrypoint.sh

الصورة كتجمع التطبيق، وCompose كيربطه بقاعدة البيانات.

backend/Dockerfile

الصورة كتجمع التطبيق، وCompose كيربطه بقاعدة البيانات.

backend/migrations/20261009112905-create-stores.js

Migration create-stores: up و down كيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. مع الأعمدة والقيود المذكورة فالمقتطف كتخلق جدول stores.

backend/migrations/20261009113405-create-users.js

Migration create-users: up و down كيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق كيرجعو. مع الأعمدة والقيود المذكورة فالمقتطف جدول users.

backend/migrations/20261009113746-create-refresh-tokens.js

Migration create-refresh-tokens: up و down كيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول refresh-tokens فالمقتطف والقيود المذكورة الأعمدة مع. كيرجعو.

backend/migrations/20261009113906-create-categories.js

Migration create-categories: up و down كيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. مع الأعمدة والقيود المذكورة فالمقتطف كتخلق جدول categories.

backend/migrations/20261009114031-create-products.js

Migration create-products: up و down كيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. مع الأعمدة والقيود المذكورة فالمقتطف كتخلق جدول products.

backend/migrations/20261009114124-create-customers.js

Migration create-customers: up و down كيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. مع الأعمدة والقيود المذكورة فالمقتطف كتخلق جدول customers.

47 • قاموس ملفات المشروع • تابع

backend/migrations/20261009114217-create-orders.js

Migration create-orders: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتخلق جدول orders فالمقتطف المذكورة والقيود الأعمدة مع.

backend/migrations/20261009114319-create-order-items.js

Migration create-order-items: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتخلق جدول order-items فالمقتطف المذكورة والقيود الأعمدة مع.

backend/migrations/20261009114433-create-conversations.js

Migration create-conversations: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتخلق جدول conversations فالمقتطف المذكورة والقيود الأعمدة مع.

backend/migrations/20261009114614-create-messages.js

Migration create-messages: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتخلق جدول messages فالمقتطف المذكورة والقيود الأعمدة مع.

backend/migrations/20261009114719-create-documents.js

Migration create-documents: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتخلق جدول documents فالمقتطف المذكورة والقيود الأعمدة مع.

backend/migrations/20261009114820-create-chunks.js

Migration create-chunks: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتخلق جدول chunks فالمقتطف المذكورة والقيود الأعمدة مع.

backend/migrations/20261009114916-create-agent-logs.js

Migration create-agent-logs: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتخلق جدول agent-logs فالمقتطف المذكورة والقيود الأعمدة مع.

backend/migrations/20261011100000-align-model-constraints.js

Migration align-model-constraints: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتعدل uniqueness وربط OrderItem بCustomer/Order؛ migration الواحد المستخدم قيد كتراجع بعدها اللي لكل متجر.

backend/migrations/20261012130000-add-role-to-users.js

Migration add-role-to-users: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتضيف role وكتحيد users_storeId_unique المتجر لنفس يدخلو حسابات عدة باش.

backend/migrations/20261017140000-add-agent-log-traces.js

Migration add-agent-log-traces: up و downكيطبق تغيير بنية القاعدة. كتتقرا حسب ترتيب التاريخ فاسم الملف. كيرجعو. كتضيف metadata الملف. trace ديال والمدة والمصادر والأدوات للاستهلاك.

47 • قاموس ملفات المشروع • تابع

backend/migrations/20261019090000-add-document-error-message.js

Migration add-document-error-message: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ باش الإدارة تشوف فشل ingestion. errorMessage فاسم الملف. كتزيد

backend/migrations/20261019150000-add-image-url-to-products.js

Migration add-image-url-to-products: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ اختياري للمنتوج imageUrl فاسم الملف. كتزيد.

backend/n8n/shopmate-manager-notifications.json

إشعار المسؤول عملية خارجية اختيارية، منفصلة على نجاح معاملة الشراء.

backend/package.json

اختبارات scripts وماشي جميع أسماء scripts، كيشغل الأمر المسجل تحت npm run.

backend/sql/001_init_pgvector.sql

البحث كيقارن تمثيل السؤال العددي بتمثيلات المقاطع داخل متجر واحد.

backend/src/app.ts

هاد الملف كيبني تطبيق Express ويربط كل مجموعة URLs بالـrouter ديالها.

backend/src/config/cors.ts

كيقرأ origins المسموحين؛ الطلب بلا Origin مسموح لـCORS. native/curl. ماشي بديل JWT.

نقاط الدخول: export const corsOptions: CorsOptions = L13:

backend/src/config/database.ts

كيخلق Sequelize PostgreSQL connection من logging:false. DB_HOST/PORT/NAME/USER/PASSWORD كيوقف طباعة SQL الاعتيادي.

backend/src/config/demo.ts

قيم تعريف متجر وحساب الديمو اللي كيستعملهم auth وseed؛ هادي اختيارات خادم، ماشي متجر يختارو العميل فـbody.

نقاط الدخول: export const DEMO_STORE_NAME = "ShopMate Demo Store";; L2: export const L1: ;"DEMO_ADMIN_EMAIL = "admin@shopmate-demo.local

backend/src/config/jwt.ts

Access token بطاقة موقعة للاستعمال مع الطلبات. Refresh token سر عشوائي لتجديد الجلسة.

نقاط الدخول: L10: export const REFRESH_TOKEN_TTL_MS: number = 7 * 24 * 60 * 60 * 1000;; L12: export interface JwtPayload; L19: export function signAccessToken(payload: JwtPayload): string; L25: export function verifyAccessToken(token: string): JwtPayload; L29: export function generateRefreshToken(): string; L33: export function hashToken(token: string): string

47 • قاموس ملفات المشروع • تابع

backend/src/config/sequelize-cli.js

إعداد خاص بـSequelize CLI باش migrations تلقى نفس قاعدة البيانات حسب البيئة.

backend/src/docs/swagger.ts

وصف OpenAPI وSwagger UI للمسارات والمدخلات والأجوبة. التوثيق ما كيشغلش قواعد البيع؛ خاص يوافق routes.

نقاط الدخول: L4: export const swaggerSpec =

backend/src/mcp/shipmentServer.ts

getShipmentStatusTool كيتأكد من صاحب الطلب قبل التواصل مع خدمة التتبع.

نقاط الدخول: L72: async function shutdown(): Promise<void>

backend/src/middlewares/auth.ts

Middleware دالة كتدخل بين الطلب والتنفيذ. إما تكمل بـnext، وإما تجاوب وترجع.

نقاط الدخول: L4: export function authenticate(

backend/src/middlewares/errorHandler.ts

كيحول الأخطاء لأجوبة HTTP: JSON خاطئ400، JWT401، duplicate409، و500 عام. console logging هنا منفصل عن sanitizeLogs.

نقاط الدخول: L4: export function errorHandler(

backend/src/middlewares/index.ts

Barrel exports: كيجمع تصدير middlewares باش imports يكونو مختصرين. فالمسارات.

backend/src/middlewares/rateLimiter.ts

حد عام افتراضي100 طلب/15 دقيقة، وحد AI20/دقيقة حسب userId. default memory store ماشي عداد موزع.

نقاط الدخول: L6: export const globalRateLimiter = rateLimit(; L18: export function createRateLimiter(options:; L39: export const agentRateLimiter = rateLimit)

backend/src/middlewares/roles.ts

Middleware دالة كتدخل بين الطلب والتنفيذ. إما تكمل بـnext، وإما تجاوب وترجع.

نقاط الدخول: L4: export function requireRole(...allowedRoles: UserRole[])

backend/src/middlewares/sanitizeLogs.ts

السجل مخصص لتتبع دور المساعد، والرسائل مخصصة لتاريخ الحوار.

نقاط الدخول: L18: export function sanitizeForLogging(data: unknown): unknown

backend/src/middlewares/tenant.ts

Middleware دالة كتدخل بين الطلب والتنفيذ. إما تكمل بـnext، وإما تجاوب وترجع.

نقاط الدخول: L6: export function injectTenant(; L37: export async function resolveStoreTenant)

47 • قاموس ملفات المشروع • تابع

backend/src/middlewares/validate.ts

الطلب الخارجي ما نثقوش فالشكل ديالو. Zod كيرجع بيانات مقبولة أو أخطاء قبل الوصول للخدمة.

نقاط الدخول: export function validate :L4)

backend/src/models/AgentLog.ts

أثر دور AI: السؤال والجواب والأدوات والمصادر والاستهلاك والمدة والرفض. ماشي نفس Message.

الحقول: id, storeId, agentName, action, status, conversationId, messageId, toolsCalled, ragSources, tokenUsage, latencyMs, refusalCategory, input, output, errorMessage, createdAt, updatedAt

backend/src/models/Category.ts

صنف منتجات تابع لمتجر، عندو الاسم والوصف. Product كيشير ليه بcategoryId.

الحقول: id, storeId, name, description, createdAt, updatedAt

backend/src/models/Chunk.ts

نص مقطع تابع لوثيقة، مع embedding VECTOR(1536) أو NULL. المقاطع بلا embedding مستبعدة من retrieval.

الحقول: id, storeId, documentId, content, embedding, createdAt, updatedAt

backend/src/models/Conversation.ts

حوار تابع لمستخدم ومتجر، بعنوان وحالة. هو الرابط بين الرسائل.

الحقول: id, storeId, userId, title, status, createdAt, updatedAt

backend/src/models/Customer.ts

هوية تجارية للزبون فالطلبات. كتتحل من User بواسطة البريد والمتجر فالمسارات الحالية.

الحقول: id, storeId, name, email, phone, createdAt, updatedAt

backend/src/models/Document.ts

metadata الاسم :ديال الوثيقة، uploader، mimeType، status، errorMessage وfileUrl وصفي.

الحقول: id, storeId, uploadedBy, name, fileUrl, mimeType, status, errorMessage, createdAt, updatedAt

backend/src/models/index.ts

كتربط النماذج ببعضياتها. Association كتبدل بنية قاعدة البيانات. Migration كيوصف التعامل مع سجل فالكود Model.

backend/src/models/Message.ts

رسالة داخل محادثة: role وcontent. واجهة الشات كتستعملها باش ترجع التاريخ.

الحقول: id, storeId, conversationId, role, content, createdAt, updatedAt

47 • قاموس ملفات المشروع • تابع

backend/src/models/Order.ts

رأس الطلب: storeId وcustomerId والحالة والمجموع. ما فيهش عمود عنوان الشحن فهاد النسخة.

الحقول: id, storeId, customerId, status, total, createdAt, updatedAt

نقاط الدخول: L4: export type OrderStatus =; L11: export interface OrderData

backend/src/models/OrderItem.ts

سطر شراء: orderId وproductId وcustomerId والكمية وunitPrice. كيحفظ الثمن وقت الشراء.

الحقول: id, storeId, orderId, customerId, productId, quantity, unitPrice, createdAt, updatedAt

backend/src/models/Product.ts

منتوج بثمن ومخزون وSKU وصورة URL وصنف اختياري. المصدر الحي للثمن والمخزون.

الحقول: id, storeId, categoryId, name, description, imageUrl, price, sku, stock, createdAt, updatedAt

backend/src/models/RefreshToken.ts

hash ديال refresh مع userId وstoreId وexpiresAt. الحذف كيبطل التجديد بذلك التوكن.

الحقول: id, storeId, userId, token, expiresAt, createdAt, updatedAt

backend/src/models/Store.ts

المتجر الجذر. id ديالو كيربط بيانات المتجر. جدول Store نفسه ما خاصوش storeId كمرجع لنفسه.

الحقول: id, name, createdAt, updatedAt

backend/src/models/tenantScope.ts

كتربط النماذج ببعضياتها Association. كتبدل بنية قاعدة البيانات Migration. كيوصف التعامل مع سجل فالكود Model.

نقاط الدخول: L5: function validateStoreId(storeId: number): number; L13: export function addStoreScope(model: TenantModel): void; L21: export function forStore(model: TenantModel, storeId: number): TenantModel

backend/src/models/User.ts

حساب الدخول: الاسم والبريد وhash كلمة السر والدور. uniqueness ديال البريد داخل المتجر، ماشي ضروري عبر جميع المتاجر.

الحقول: id, storeId, name, email, password, role, createdAt, updatedAt

نقاط الدخول: L4: export type UserRole = "ADMIN" | "CUSTOMER";

backend/src/routes/agent.ts

هاد route هو المنظم: هوية → محادثة → حمايات → مصادر → نموذج وأدوات → جواب وسجل.

47 • قاموس ملفات المشروع • تابع

backend/src/routes/auth.ts

الواجهة كترسل الاسم والبريد وكلمة السر، والخادم هو اللي كيحدد المتجر والدور.

نقاط الدخول: L27: async function issueTokenPair(user: User): Promise<

backend/src/routes/categories.ts

الإدارة كتحتاج التحقق من الدور والمتجر والمدخلات، حتى إلا الواجهة عطلات الأزرار لغير المسؤول.

backend/src/routes/documents.ts

الملف المرفوع كيتحول لنص، ثم مقاطع، ثم embeddings، ثم سجلات قابلة للبحث.

backend/src/routes/orders.ts

POST /api/orders كيحتاج confirmed:true وكيحسب الثمن من القاعدة.

نقاط الدخول: L23: async function resolveCustomer(storeId: number, email: string, name?: string)

backend/src/routes/products.ts

هاد المسار كيحول الفلاتر اللي فالواجهة لشروط استعلام محددة.

backend/src/schemas/agentTools.ts

هاد الطبقة كتحدد شنو يقدر المساعد يدير وبأي مدخلات.

نقاط الدخول: L10: export const searchProductsToolSchema = z.object(; L18: export const getProductDetailsToolSchema = z.object(; L23: export const checkStockToolSchema = z.object(; L28: export const createOrderToolSchema = z.object(; L44: export const getOrderStatusToolSchema = z.object(; L49: export const listMyOrdersToolSchema = z.object(; L54: export const cancelOrderToolSchema = z.object(; L60: export const getShipmentStatusToolSchema = z

backend/src/schemas/auth.ts

الطلب الخارجي ما نثقوش فالشكل ديالو. Zod كيرجع بيانات مقبولة أو أخطاء قبل الوصول للخدمة.

نقاط الدخول: L3: export const registerSchema = z.object(; L20: export const loginSchema = z.object(; L31: export const refreshSchema = z.object(; L38: export const logoutSchema = z.object(; L45: export type RegisterInput = z.infer<typeof registerSchema>;; L46: export type LoginInput = z.infer<typeof loginSchema>;; L47: export type RefreshInput = z.infer<typeof refreshSchema>;; L48: export type LogoutInput = z.infer<typeof logoutSchema>;

47 • قاموس ملفات المشروع • تابع

backend/src/schemas/category.ts

الطلب الخارجي ما نثقوش فالشكل ديالو. Zod كيرجع بيانات مقبولة أو أخطاء قبل الوصول للخدمة.

نقاط الدخول: L3: export const categoryIdParamSchema = z.object(; L7: export const createCategorySchema = z.object(; L16: export const updateCategorySchema = z.object(; L26: export type CategoryIdParam = z.infer<typeof categoryIdParamSchema>;; L27: export type CreateCategoryInput = z.infer<typeof createCategorySchema>;; L28: export type UpdateCategoryInput = z.infer<typeof updateCategorySchema>;

backend/src/schemas/order.ts

الطلب الخارجي ما نثقوش فالشكل ديالو. Zod كيرجع بيانات مقبولة أو أخطاء قبل الوصول للخدمة.

نقاط الدخول: L3: export const orderItemInputSchema = z.object(; L8: export const createOrderSchema = z.object(; L18: export const orderConfirmationSchema = z.object(; L22: export const orderIdParamSchema = z.object(; L26: export const updateOrderStatusSchema = z.object(; L36: export type CreateOrderInput = z.infer<typeof createOrderSchema>;; L37: export type OrderIdParam = z.infer<typeof orderIdParamSchema>;; L38: export type UpdateOrderStatusInput = z.infer<typeof updateOrderStatusSchema>;

backend/src/schemas/product.ts

الطلب الخارجي ما نثقوش فالشكل ديالو. Zod كيرجع بيانات مقبولة أو أخطاء قبل الوصول للخدمة.

نقاط الدخول: L3: export const productQuerySchema = z.object(; L13: export const productIdParamSchema = z.object(; L17: export const createProductSchema = z.object(; L43: export const updateProductSchema = createProductSchema.partial();; L45: export type ProductQueryInput = z.infer<typeof productQuerySchema>;; L46: export type ProductIdParam = z.infer<typeof productIdParamSchema>;; L47: export type CreateProductInput = z.infer<typeof createProductSchema>;; L48: export type UpdateProductInput = z.infer<typeof updateProductSchema>;

backend/src/scripts/applyPgvectorSql.ts

هنا كنجهزو البيانات والبنية باش API تلقى شنو تخدم عليه.

نقاط الدخول: L9: async function main(): Promise<void>

backend/src/scripts/test-agent-cost-limits.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L7: function assert(name: string, condition: boolean): void; L12: async function run(): Promise<void>

backend/src/scripts/test-agent-tools.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L19: async function runAgentToolsTest()

47 • قاموس ملفات المشروع • تابع

backend/src/scripts/test-auth-tenant.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L29: async function start(app: express.Express); L40: async function request(baseUrl: string, path: string, init: RequestInit =; L47: async function testAuthRotationAndLogout(): Promise<void>; L120: <async function testTenantAndOrderIsolation(): Promise<void>; L184: async function main(): Promise<void

backend/src/scripts/test-guardrails.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L14: function assert(name: string, condition: boolean): void

backend/src/scripts/test-llm.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L21: async function runLlmStreamTest()

backend/src/scripts/test-n8n-notifications.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L5: async function main(): Promise<void>

backend/src/scripts/test-order-confirmation.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L32: async function runOrderConfirmationTest()

backend/src/scripts/test-read-tools.ts

Arrange تجهيز الحالة، Act تنفيذ العملية، Assert مقارنة النتيجة بالمتوقع.

نقاط الدخول: L27: async function runReadToolsTest()

backend/src/seed.ts

وجود Document ماشي هو وجود معرفة مفهرسة وقابلة للاسترجاع.

نقاط الدخول: L65: async function seed(): Promise<void>

backend/src/server.ts

اتصال قاعدة البيانات والشبكة كيحتاجو وقت. Promise كتمثل نتيجة غادي تجي من بعد.

نقاط الدخول: L8: async function startServer()

47 • قاموس ملفات المشروع • تابع

backend/src/services/agentTools.ts

اقتراح الأداة من النموذج كيرجع للخادم؛ الخادم ينفذ ويرجع النتيجة للنموذج.

نقاط الدخول: L29: export const ALLOWED_TOOLS = [; L40: export type AllowedToolName = (typeof ALLOWED_TOOLS)[number];; L42: export function isAllowedTool(name: string): name is AllowedToolName; L49: export const MAX_TOOL_CALLS_PER_TURN = 3;; L51: export function canCallAnotherTool(toolCallsUsed: number): boolean; L65: export interface ToolExecutionContext; L75: export const TOOL_SCHEMAS: Record<AllowedToolName, z.ZodTypeAny> =; L89: export const AGENT_TOOLS: LLMTool[] =

backend/src/services/guardrails.ts

هاد الملف فيه قواعد نصية للرفض وتنقية السياق ومراجعة الجواب.

نقاط الدخول: L17: export interface InjectionCheckResult; L23: export interface RefusalCheckResult; L29: export interface OutputModerationResult; L64: function detectInjectionPattern(input: string): RegExp | undefined; L72: export function checkPromptInjection(input: string): InjectionCheckResult; L94: export function sanitizeContextData(text: string): string; L180: export const STANDARD_REFUSALS =; L198: export function checkProhibitedIntent(text: string): RefusalCheckResult

backend/src/services/llm.ts

هاد الملف كيجمع إعدادات الاتصال وتعليمات المساعد وآلية قراءة streaming من المزود.

نقاط الدخول: L13: export function getLlmBaseUrl(): string | undefined; L22: export function getLlmApiKey(): string; L31: export function getLlmClient(): OpenAI; L40: export const LLM_MODEL =; L44: export const MAX_TOKENS_PER_TURN = 1024;; L60: export function buildSystemPrompt(storeName: string = "ShopMate"): string; L104: export type LLMMessage = ChatCompletionMessageParam;; L105: export type LLMTool = ChatCompletionTool;

backend/src/services/mcpShipmentClient.ts

getShipmentStatusTool كيتأكد من صاحب الطلب قبل التواصل مع خدمة التتبع.

نقاط الدخول: L16: function fetchWithTimeout(input: string | URL | Request, init?: RequestInit): Promise<Response>; L31: export async function requestShipmentStatus(

backend/src/services/n8nNotifications.ts

إشعار المسؤول عملية خارجية اختيارية، منفصلة على نجاح معاملة الشراء.

نقاط الدخول: L9: export interface OrderCreatedNotification; L30: export interface ManagerRequestNotification; L44: async function sendToN8n(payload: NotificationPayload): Promise<boolean>; L73: export function notifyOrderCreated(notification: OrderCreatedNotification): Promise<boolean>; L77: export function notifyManagerRequest(notification: ManagerRequestNotification): Promise<boolean>

47 • قاموس ملفات المشروع • تابع

backend/src/services/orderConfirmation.ts

كتقرا المعطيات وتحضر ملخص، وتخزن نية العملية مؤقتاً initiateCreateOrderRecap.

نقاط الدخول: L9: export interface PendingOrderAction; L34: export function isAffirmativeConfirmation(text: string): boolean; L76: export function hasPendingConfirmation(conversationId: number): boolean; L89: export function getPendingConfirmation(; L108: export async function initiateCreateOrderRecap(; L199: export async function initiateCancelOrderRecap(; L262: export async function executeConfirmedAction(; L278: async function performConfirmedAction()

backend/src/services/rag.ts

هاد الدالة كتخدم بالنص فقط، بلا database وبلا LLM.

نقاط الدخول: L26: export function chunkText(text: string): string[]; L76: export async function embedText(text: string): Promise<number[]>; L88: export async function embedChunks(chunks: string[]): Promise<number[][]>; L112: export interface RetrievedChunk; L128: export const SIMILARITY_THRESHOLD = 0.75;; L143: export async function retrieveContext()

backend/src/types/express.d.ts

توسيع TypeScript لأنواع Express باش req.user وreq.storeId يكونو معروفين للمحلل؛ ما كيعمرهمش وقت التشغيل.

الحقول: global

backend/tsconfig.json

كيفاش TypeScript ديال الخادم كيتحول لJavaScript ومكان الخرج.

docker-compose.yml

الصورة كتجمع التطبيق، وCompose كيربطه بقاعدة البيانات.

mobile/app/(auth)/_layout.tsx

كيتابع حالة الجلسة RootLayoutكيربط بنية الملفات بالشاشات، و Expo Router.

نقاط الدخول: L3: export default function AuthLayout()

mobile/app/(auth)/login.tsx

Component دالة كتنتج واجهة. State و propsبيانات كتبدل العرض، و component آخر مدخلات جاية من.

نقاط الدخول: L23: export default function LoginScreen()

mobile/app/(auth)/register.tsx

ملف مساعد داخل (auth). قرا declarations والمقتطف باش تربط الدور ديالو بالمسار المشروح قبل.

نقاط الدخول: L23: export default function RegisterScreen()

47 • قاموس ملفات المشروع • تابع

mobile/app/(tabs)/_layout.tsx

كيتابع حالة الجلسة RootLayoutكيربط بنية الملفات بالشاشات، و Expo Router.

نقاط الدخول: L5: export default function TabLayout()

mobile/app/(tabs)/admin-agent-logs.tsx

كل شاشة كتراجع ADMIN فالواجهة وكتتواصل مع endpoints محمية فالخادم.

نقاط الدخول: L40: export default function AdminAgentLogsScreen(); L167: function LogCard(; L179: function TurnDetail(; L202: function DetailSection(; L206: function Badge(; L212: function FilterChip(; L216: function displayValue(value: unknown): string; L222: function formatDate(value: string): string

mobile/app/(tabs)/admin-catalogue.tsx

الفورموليز كيحتافظ بالنصوص، وكيحول الأرقام قبل الإرسال.

نقاط الدخول: L27: export default function AdminCatalogScreen(); L198: function Field(; L208: function TabButton(; L212: function Chip(; L216: function TextButton()

mobile/app/(tabs)/admin-documents.tsx

كل شاشة كتراجع ADMIN فالواجهة وكتتواصل مع endpoints محمية فالخادم.

نقاط الدخول: L30: export default function AdminDocumentsScreen(); L169: function statusLabel(status: RagDocument["status"]): string

mobile/app/(tabs)/chat.tsx

الشاشة كتدير state ديال المحادثة، بينما الخادم هو اللي كينفذ الخدمات.

نقاط الدخول: L32: export interface ProductCardData; L41: export interface OrderRecapData; L49: export interface CreatedOrderData; L55: export interface ChatMessage; L77: function formatConversationDate(dateStr: string): string; L96: export default function ChatScreen()

mobile/app/(tabs)/index.tsx

الشاشة كتعرض وتجمع الاختيارات، والـstore كيرسل الفلاتر وكيحتافظ بالنتائج.

نقاط الدخول: L32: export default function CatalogScreen(); L154: function CatalogHeader(; L267: function ProductCard()

mobile/app/(tabs)/orders.tsx

كل شاشة كتربط state وعناصر العرض بدالة أو API واضحة.

نقاط الدخول: L48: function getStatusStepIndex(status: string): number; L63: function getStatusBadge(status: string); L80: export default function OrdersScreen(); L203: function OrderCard()

47 • قاموس ملفات المشروع • تابع

mobile/app/(tabs)/profile.tsx

كل شاشة كتربط state وعناصر العرض بدالة أو API واضحة.

نقاط الدخول: L24: export default function ProfileScreen(); L166: function MenuItem()

mobile/app/+html.tsx

كل شاشة كتربط state وعناصر العرض بدالة أو API واضحة.

نقاط الدخول: L4: export default function Root()

mobile/app/+not-found.tsx

كل شاشة كتربط state وعناصر العرض بدالة أو API واضحة.

نقاط الدخول: L5: export default function NotFoundScreen()

mobile/app/_layout.tsx

كيتابع حالة الجلسة RootLayoutكيربط بنية الملفات بالشاشات، و Expo Router.

نقاط الدخول: L14: function SplashScreen(); L31: export default function RootLayout()

mobile/app/modal.tsx

كل شاشة كتربط state وعناصر العرض بدالة أو API واضحة.

mobile/app/product/[id].tsx

الشاشة كتعرض وتجمع الاختيارات، والـstore كيرسل الفلاتر وكيحتافظ بالنتائج.

نقاط الدخول: L30: export default function ProductDetailScreen()

mobile/app.json

إعداد Expo: الاسم والأيقونة وsplash والمنصات والpluginsماشي كود معالجة طلبات.

mobile/components/index.ts

نقطة export للمكونات حسب الملف الحالي، باش تسهل imports.

mobile/constants/Colors.ts

تعريف ألوان إضافي. تحقق واش الشاشة كتستعمل هاد الملف أو theme/tokens قبل أي تعديل لون.

mobile/package.json

اختبارات scripts وماشي جميع أسماء scripts، كيشغل الأمر المسجل تحت npm run.

47 • قاموس ملفات المشروع • تابع

mobile/src/api/agentChat.ts

الاتصال واحد، والخادم كيكتب فيه أحداث JSON مفصولة بسطرين.

نقاط الدخول: L5: export interface ChatSource; L10: export interface StreamEventPayload; L32: export interface StreamChatCallbacks; L51: export interface ActiveStream; L60: export async function streamAgentChat(; L204: export interface ConversationSummary; L219: export async function fetchConversationHistory(): Promise<ConversationSummary[]>; L229: export async function fetchSingleConversation(id: number): <Promise<ConversationSummary

mobile/src/api/client.ts

باش ما نكتبوش Authorization ومعالجة401 فكل شاشة، client كيجمعهم.

نقاط الدخول: L5: export const apiClient = axios.create(; L19: export function setAuthCallbacks(callbacks:

mobile/src/auth/tokenStorage.ts

كيجمع حالة الجلسة والدوال اللي كتغيرها فبلاصة واحدة Zustand.

نقاط الدخول: L7: export interface StoredTokens; L12: export async function saveTokens(tokens:; L30: export async function getTokens(): Promise<StoredTokens>; L50: export async function clearTokens(): <Promise<void

mobile/src/components/BrandHeader.tsx

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

نقاط الدخول: L5: export function BrandHeader()

mobile/src/components/Button.tsx

مع الكود React كيخلينا نكتب واجهة TSX. هو السلوك اللي كيتنفذ JavaScriptكيوصف البيانات، و TypeScript.

نقاط الدخول: L12: export type ButtonVariant = "primary" | "secondary" | "outline" | "danger" | "ghost";; L13: export type ButtonSize = "sm" | "md" | "lg";; L15: export interface ButtonProps; L27: export function Button(

mobile/src/components/Card.tsx

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

نقاط الدخول: L5: export interface CardProps extends ViewProps; L11: export function Card(

mobile/src/components/EmptyState.tsx

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

نقاط الدخول: L6: export interface EmptyStateProps; L15: export function EmptyState(

mobile/src/components/ErrorState.tsx

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

نقاط الدخول: L6: export interface ErrorStateProps; L13: export function ErrorState(

47 • قاموس ملفات المشروع • تابع

mobile/src/components/index.ts

كيجمع exports ديال المكونات المشتركة.

mobile/src/components/Input.tsx

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

نقاط الدخول: L12: export interface InputProps extends TextInputProps; L20: export function Input(

mobile/src/components/Loader.tsx

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

نقاط الدخول: L5: export interface LoaderProps; L12: export function Loader(

mobile/src/config/api.ts

باش ما نكتبوش Authorization ومعالجة401 فكل شاشة، client كيجمعهم.

نقاط الدخول: L9: export const API_BASE_URL: string =

mobile/src/stores/authStore.ts

Zustand كيجمع حالة الجلسة والدوال اللي كتغيرها فبلاصة واحدة.

نقاط الدخول: L5: export interface AuthUser; L29: export const useAuthStore = create<AuthState>((set, get) =>

mobile/src/stores/catalogStore.ts

الشاشة كتعرض وتجمع الاختيارات، والـstore كيرسل الفلاتر وكيحتافظ بالنتائج.

نقاط الدخول: L4: export interface Category; L10: export interface Product; L56: export const useCatalogStore = create<CatalogState>((set, get) =>)

mobile/src/theme/tokens.ts

القيم المشتركة كتخلي الشاشات تستعمل نفس اللغة البصرية.

نقاط الدخول: L1: export const colorTokens =; L40: export const spacingTokens =; L49: export const radiusTokens =; L58: export const typographyTokens =

mobile/tsconfig.json

قواعد تحليل TypeScript وaliasالمسارات فالموبايل.

Model • Store

المتجر الجذر. id ديالو كيربط بيانات المتجر. جدول Store نفسه ما خاصوش storeId كمرجع لنفسه.

backend/src/models/Store.ts

backend/src/models/Store.ts

```
26  Store.init(
27    {
28      id: {
29        type: DataTypes.INTEGER,
30        autoIncrement: true,
31        primaryKey: true,
32      },
33
34      name: {
35        type: DataTypes.STRING,
36        allowNull: false,
37      },
38    },
39    {
40      sequelize,
41      tableName: "stores",
42      timestamps: true,
43    }
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Store فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • User

حساب الدخول: الاسم والبريد وhash كلمة السر والدور. uniqueness ديال البريد داخل المتجر، ماشي ضروري عبر جميع المتاجر.

backend/src/models/User.ts

backend/src/models/User.ts

```
30  User.init(
31    {
32      id: {
33        type: DataTypes.INTEGER,
34        autoIncrement: true,
35        primaryKey: true,
36      },
37
38      storeId: {
39        type: DataTypes.INTEGER,
40        allowNull: false,
41      },
42
43      name: {
44        type: DataTypes.STRING,
45        allowNull: false,
46      },
47
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم User فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • RefreshToken

hash ديال refresh مع userId وstoreId وexpiresAt. الحذف كيبطل التجديد بذلك التوكن.

backend/src/models/RefreshToken.ts

backend/src/models/RefreshToken.ts

```
28  RefreshToken.init({
29    id: {
30      type: DataTypes.INTEGER,
31      autoIncrement: true,
32      primaryKey: true,
33    },
34    userId: {
35      type: DataTypes.INTEGER,
36      allowNull: false,
37    },
38    storeId: {
39      type: DataTypes.INTEGER,
40      allowNull: false,
41    },
42    token: {
43      type: DataTypes.STRING,
44      allowNull: false,
45    },
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم RefreshToken فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Category

صنف منتجات تابع لمتجر، عندو الاسم والوصف. Product كيشير ليه بـcategoryId.

backend/src/models/Category.ts

backend/src/models/Category.ts

```
24  Category.init(
25    {
26      id: {
27        type: DataTypes.INTEGER,
28        autoIncrement: true,
29        primaryKey: true,
30      },
31
32      storeId: {
33        type: DataTypes.INTEGER,
34        allowNull: false,
35      },
36
37      name: {
38        type: DataTypes.STRING,
39        allowNull: false,
40      },
41
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وكيوصف tableName. declare كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Category فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Product

منتوج بثمن ومخزون وSKU وصورة URL وصنف اختياري. المصدر الحي للثمن والمخزون.

backend/src/models/Product.ts

backend/src/models/Product.ts

```
37  Product.init(
38    {
39      id: {
40        type: DataTypes.INTEGER,
41        autoIncrement: true,
42        primaryKey: true,
43      },
44      storeId: {
45        type: DataTypes.INTEGER,
46        allowNull: false,
47      },
48      categoryId: {
49        type: DataTypes.INTEGER,
50        allowNull: true,
51      },
52      name: {
53        type: DataTypes.STRING,
54        allowNull: false,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Product فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Customer

هوية تجارية للزبون فالطلبات. كتتحل من User بواسطة البريد والمتجر فالمسارات الحالية.

backend/src/models/Customer.ts

backend/src/models/Customer.ts

```
28  Customer.init(
29    {
30      id: {
31        type: DataTypes.INTEGER,
32        autoIncrement: true,
33        primaryKey: true,
34      },
35      storeId: {
36        type: DataTypes.INTEGER,
37        allowNull: false,
38      },
39      name: {
40        type: DataTypes.STRING,
41        allowNull: false,
42      },
43      email: {
44        type: DataTypes.STRING,
45        allowNull: true,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Customer فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Order

رأس الطلب: storeId وcustomerId والحالة والمجموع. ما فيهش عمود عنوان الشحن فهاد النسخة.

backend/src/models/Order.ts

backend/src/models/Order.ts

```
31  Order.init(
32    {
33      id: {
34        type: DataTypes.INTEGER,
35        autoIncrement: true,
36        primaryKey: true,
37      },
38      storeId: {
39        type: DataTypes.INTEGER,
40        allowNull: false,
41      },
42      customerId: {
43        type: DataTypes.INTEGER,
44        allowNull: false,
45      },
46      status: {
47        type: DataTypes.STRING,
48        allowNull: false,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Order فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • OrderItem

سطر شراء: orderId وproductId وcustomerId والكمية وunitPrice. كيحفظ الثمن وقت الشراء.

backend/src/models/OrderItem.ts

backend/src/models/OrderItem.ts

```
32  OrderItem.init(
33    {
34      id: {
35        type: DataTypes.INTEGER,
36        autoIncrement: true,
37        primaryKey: true,
38      },
39      storeId: {
40        type: DataTypes.INTEGER,
41        allowNull: false,
42      },
43      orderId: {
44        type: DataTypes.INTEGER,
45        allowNull: false,
46      },
47      customerId: {
48        type: DataTypes.INTEGER,
49        allowNull: false,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم OrderItem فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Conversation

حوار تابع لمستخدم ومتجر، بعنوان وحالة. هو الرابط بين الرسائل.

backend/src/models/Conversation.ts

backend/src/models/Conversation.ts

```
28  Conversation.init(
29    {
30      id: {
31        type: DataTypes.INTEGER,
32        autoIncrement: true,
33        primaryKey: true,
34      },
35      storeId: {
36        type: DataTypes.INTEGER,
37        allowNull: false,
38      },
39      userId: {
40        type: DataTypes.INTEGER,
41        allowNull: false,
42      },
43      title: {
44        type: DataTypes.STRING,
45        allowNull: false,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Conversation فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Message

رسالة داخل محادثة: role وcontent. واجهة الشات كتستعملها باش ترجع التاريخ.

backend/src/models/Message.ts

backend/src/models/Message.ts

```
30  Message.init(
31    {
32      id: {
33        type: DataTypes.INTEGER,
34        autoIncrement: true,
35        primaryKey: true,
36      },
37      storeId: {
38        type: DataTypes.INTEGER,
39        allowNull: false,
40      },
41      conversationId: {
42        type: DataTypes.INTEGER,
43        allowNull: false,
44      },
45      role: {
46        type: DataTypes.ENUM("user", "assistant", "system"),
47        allowNull: false,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Message فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Document

metadata ديال الوثيقة: الاسم، errorMessage، status، mimeType، uploader وfileUrl وصفي.

backend/src/models/Document.ts

backend/src/models/Document.ts

```
39  Document.init(
40    {
41      id: {
42        type: DataTypes.INTEGER,
43        autoIncrement: true,
44        primaryKey: true,
45      },
46      storeId: {
47        type: DataTypes.INTEGER,
48        allowNull: false,
49      },
50      uploadedBy: {
51        type: DataTypes.INTEGER,
52        allowNull: false,
53      },
54      name: {
55        type: DataTypes.STRING,
56        allowNull: false,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Document فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • Chunk

نص مقطع تابع لوثيقة، مع embedding VECTOR(1536) أو NULL. المقاطع بلا embedding مستبعدة من retrieval.

backend/src/models/Chunk.ts

backend/src/models/Chunk.ts

```
34  Chunk.init(
35    {
36      id: {
37        type: DataTypes.INTEGER,
38        autoIncrement: true,
39        primaryKey: true,
40        allowNull: false,
41      },
42
43      storeId: {
44        type: DataTypes.INTEGER,
45        allowNull: false,
46      },
47
48      documentId: {
49        type: DataTypes.INTEGER,
50        allowNull: false,
51      },
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم Chunk فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

Model • AgentLog

أثر دور AI: السؤال والجواب والأدوات والمصادر والاستهلاك والمدة والرفض. ماشي نفس Message.

backend/src/models/AgentLog.ts

backend/src/models/AgentLog.ts

```
AgentLog.init(
  {
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true,
    },
    storeId: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
    agentName: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    action: {
      type: DataTypes.STRING,
      allowNull: false,
```

كيفاش تقرا هاد التعريف؟

اسم الخاصية هو العمود. type النوع، allowNull واش الفراغ مسموح، defaultValue قيمة تلقائية، primaryKey معرف فريد. init كيربط التعريف بـsequelize وdeclare. tableName كيوصف TypeScript، وinit كيوصف ORM وقت التشغيل.

ربط بالخدمة

قلب على اسم AgentLog فـroutes أو services. findOne/findAll قراءة، create/bulkCreate إنشاء، update تعديل وdestroy حذف. المعاملة ما كتدخلش تلقائياً لمجرد أن model كاين داخل دالة async.

48 • قراءة migrations بالترتيب

المقتطفات كتوريك العملية البنيوية. up وdown خاصهم يتراجعو معاً، وما نشغلوش down على بيانات مهمة بلا خطة.

20261009112905-create-stores.js

Migration create-stores: up و downكيطبق تغيير بنية القاعدة. كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول stores فالمقتطف مع الأعمدة والقيود المذكورة.

backend/migrations/20261009112905-create-stores.js

```
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable("stores", {
      id: {
        type: Sequelize.INTEGER,
        autoIncrement: true,
        primaryKey: true,
        allowNull: false,
      },

      name: {
        type: Sequelize.STRING,
        allowNull: false,
```

20261009113405-create-users.js

Migration create-users: up و downكيطبق تغيير بنية القاعدة. كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول users فالمقتطف مع الأعمدة والقيود المذكورة.

backend/migrations/20261009113405-create-users.js

```
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable("users", {
      id: {
        type: Sequelize.INTEGER,
        autoIncrement: true,
        primaryKey: true,
        allowNull: false,
      },

      storeId: {
        type: Sequelize.INTEGER,
        allowNull: false,
```

20261009113746-create-refresh-tokens.js

Migration create-refresh-tokens: up و downكيطبق تغيير بنية القاعدة. كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول refresh-tokens فالمقتطف مع الأعمدة والقيود المذكورة.

48 • قراءة migrations بالترتيب • تابع

backend/migrations/20261009113746-create-refresh-tokens.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("refresh_tokens", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261009113906-create-categories.js

Migration create-categories: up و down كيطبق تغيير بنية القاعدة فاسم الملف. كيرجعو. كتتقرا حسب ترتيب التاريخ فالمقتطف كتخلق جدول categories مع الأعمدة والقيود المذكورة.

backend/migrations/20261009113906-create-categories.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("categories", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261009114031-create-products.js

Migration create-products: up و down كيطبق تغيير بنية القاعدة فاسم الملف. كيرجعو. كتتقرا حسب ترتيب التاريخ فالمقتطف كتخلق جدول products مع الأعمدة والقيود المذكورة.

backend/migrations/20261009114031-create-products.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("products", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

48 • قراءة migrations بالترتيب • تابع

20261009114124-create-customers.js

Migration create-customers: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول customers فالمقتطف مع الأعمدة والقيود المذكورة.

backend/migrations/20261009114124-create-customers.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("customers", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261009114217-create-orders.js

Migration create-orders: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول orders فالمقتطف مع الأعمدة والقيود المذكورة.

backend/migrations/20261009114217-create-orders.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("orders", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261009114319-create-order-items.js

Migration create-order-items: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول order-items فالمقتطف مع الأعمدة والقيود المذكورة.

48 • قراءة migrations بالترتيب • تابع

backend/migrations/20261009114319-create-order-items.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("order_items", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261009114433-create-conversations.js

Migration create-conversations: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول conversations فالمقتطف المذكورة والقيود مع الأعمدة.

backend/migrations/20261009114433-create-conversations.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("conversations", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261009114614-create-messages.js

Migration create-messages: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول messages فالمقتطف المذكورة والقيود مع الأعمدة.

backend/migrations/20261009114614-create-messages.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("messages", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

48 • قراءة migrations بالترتيب • تابع

20261009114719-create-documents.js

Migration create-documents: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول documents فالمقتطف مع الأعمدة والقيود المذكورة.

backend/migrations/20261009114719-create-documents.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("documents", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261009114820-create-chunks.js

Migration create-chunks: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول chunks فالمقتطف مع الأعمدة والقيود المذكورة.

backend/migrations/20261009114820-create-chunks.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.sequelize.query(
6        'CREATE EXTENSION IF NOT EXISTS vector;'
7      );
8
9      await queryInterface.createTable("chunks", {
10       id: {
11         type: Sequelize.INTEGER,
12         autoIncrement: true,
13         primaryKey: true,
14         allowNull: false,
15       },
```

20261009114916-create-agent-logs.js

Migration create-agent-logs: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتخلق جدول agent-logs فالمقتطف مع الأعمدة والقيود المذكورة.

48 • قراءة migrations بالترتيب • تابع

backend/migrations/20261009114916-create-agent-logs.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.createTable("agent_logs", {
6        id: {
7          type: Sequelize.INTEGER,
8          autoIncrement: true,
9          primaryKey: true,
10         allowNull: false,
11       },
12
13       storeId: {
14         type: Sequelize.INTEGER,
15         allowNull: false,
```

20261011100000-align-model-constraints.js

Migration align-model-constraints: up و down فاسم الملف. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتعدل uniqueness وربط OrderItem بCustomer/Order؛ migration الواحد اللي بعدها كتراجع قيد المستخدم الواحد لكل متجر.

backend/migrations/20261011100000-align-model-constraints.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.removeConstraint("users", "users_email_key");
6      await queryInterface.addIndex("users", ["storeId", "email"], {
7        unique: true,
8        name: "users_storeId_email_unique",
9      });
10     await queryInterface.addIndex("users", ["storeId"], {
11       unique: true,
12       name: "users_storeId_unique",
13     });
14
15     await queryInterface.addIndex("orders", ["customerId", "id"], {
```

20261012130000-add-role-to-users.js

Migration add-role-to-users: up و down كيطبق تغيير بنية القاعدة و كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. باش عدة حسابات يدخلو لنفس المتجر users_storeId_unique وكتحيد role كتضيف.

backend/migrations/20261012130000-add-role-to-users.js

```
4    async up(queryInterface, Sequelize) {
5      // 1. Add role column to users table (default CUSTOMER)
6      await queryInterface.addColumn("users", "role", {
7        type: Sequelize.STRING,
8        allowNull: false,
9        defaultValue: "CUSTOMER",
10     });
11
12     // 2. Remove the users_storeId_unique index so multiple users can register under the same store
13     // (users_storeId_email_unique remains to enforce email uniqueness per store)
14     try {
15       await queryInterface.removeIndex("users", "users_storeId_unique");
```

48 • قراءة migrations بالترتيب • تابع

20261017140000-add-agent-log-traces.js

Migration add-agent-log-traces: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتضيف metadata ديال trace والمدة والمصادر والأدوات والاستهلاك.

backend/migrations/20261017140000-add-agent-log-traces.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.addColumn("agent_logs", "conversationId", {
6        type: Sequelize.INTEGER,
7        allowNull: true,
8        references: { model: "conversations", key: "id" },
9        onUpdate: "CASCADE",
10       onDelete: "SET NULL",
11     });
12     await queryInterface.addColumn("agent_logs", "messageId", {
13       type: Sequelize.INTEGER,
14       allowNull: true,
15       references: { model: "messages", key: "id" },
```

20261019090000-add-document-error-message.js

Migration add-document-error-message: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتزيد errorMessage باش الإدارة تشوف فشل ingestion.

backend/migrations/20261019090000-add-document-error-message.js

```
4    async up(queryInterface, Sequelize) {
5      await queryInterface.addColumn("documents", "errorMessage", {
6        type: Sequelize.STRING,
7        allowNull: true,
8      });
9    },
10
11   async down(queryInterface) {
12     await queryInterface.removeColumn("documents", "errorMessage");
13   },
14 };
```

20261019150000-add-image-url-to-products.js

Migration add-image-url-to-products: up كيطبق تغيير بنية القاعدة و down كيرجعو. كتتقرا حسب ترتيب التاريخ فاسم الملف. كتزيد imageUrl اختياري للمنتوج.

48 • قراءة migrations بالترتيب • تابع

backend/migrations/20261019150000-add-image-url-to-products.js

```
  async up(queryInterface, Sequelize) {
    await queryInterface.addColumn("products", "imageUrl", {
      type: Sequelize.STRING(2048),
      allowNull: true,
    });
  },

  async down(queryInterface) {
    await queryInterface.removeColumn("products", "imageUrl");
  },
};
```

49 • تمارين تطبيقية بأجوبة

حل بصوتك قبل ما تشوف الجواب. الأمثلة افتراضية، وما كنشغلو حتى عملية شراء هنا.

طلب quantity:-2

الرحلة: شاشة → 400← schema.quantity.min(1) ← validate ← HTTP. ما خاصناش نوصلو لـTypeScript decrement. فالتليفون ما كيمنعش شخص يرسل request يدوي.

متجر A باغي منتوج B

الجواب404 إذا ما كاينش منتوج مطابق .scope بوحدو ما كيبدلش B ديال A. ID كيضيف شرط forStore. A كيحدد JWT داخل A.

جوج طلبات على آخر قطعة

المعاملة الأولى كتقفل السجل وتقرا stock1 ثم تنقصو. الثانية كتستنى القفل ثم كتلقى القيمة الجديدة وتراجعها. خاص اختبار بـDB حقيقية باش نثبت السلوك العملي تحت التزامن.

بغيت remise

كيوقع استدعاء إشعار discount/refund كتشد الصياغات المعروفة وترجع رفض، ومع checkProhibitedIntent best-effort. ما كايناش أداة معتمدة لتغيير الثمن.

مخزون تبدل بين recap وOui

التنفيذ كيعيد قراءة stock بقفل. إذا ما بقا كافي، المعاملة كتفشل بدل الاعتماد على الصورة القديمة فالملخص.

شنو تكتب فالjournal؟

كتب بكلامك أنك ربطت الواجهة بالمسارات وبالقاعدة، فهمت الفرق بين RAG والأدوات، ولقيت حدود فالتأكيد والذاكرة والتوثيق. ما تكتبش أنك شغلت اختبارات ما شغلتهاش.

عمر، نبداو من الصفر

ShopMate AI • دليل الفهم بالدارجة • التحضير للمناقشة ديال 22 أكتوبر 2026

هاد الدليل علاش؟

الهدف هو تفهم شنو بنيتي، علاش كل جزء كاين، وكيفاش الأجزاء كيخدمو مع بعضياتهم. ماشي ضروري تحفظ الكود كامل اليوم. خاصك تبني صورة واضحة، ومن بعد تربطها بالملفات.

كيفاش تقراه؟

قرا درس واحد. سد الدليل. شرح الفكرة بصوتك باستعمال مثال. إلا تعطلتي، رجع غير للفقرة اللي ما فهمتيش. كل درس فيه سؤال صغير؛ الأجوبة كاينة فالآخر.

المثال اللي غادي نتبعو

زبون بغا يشري جوج تيشورتات. فالمثال فقط: الثمن 120 درهم للوحدة والمخزون 5. هادو أرقام تعليمية، ماشي بيانات حقيقية من المتجر ديالك.

شنو راجعنا؟

الشرح مبني على README وملفات الكود المحلية ديال المشروع نهار 20 أكتوبر 2026. ماشي شهادة أن النشر خدام أو أن جميع الاختبارات دازت. الكود يقدر يتبدل من بعد.

أول فكرة تحفظها بالفهم

تطبيق ديال البيع بالدفع عند التسليم. الزبون يقدر يشوف المنتجات ويسول المساعد ShopMate ويطلب. المسؤول كيسير الكاتالوغ والوثائق وكيشوف أثر تفاعلات المساعد.

خريطة القراية

ما تقراش كلشي دفعة وحدة. هاد المراحل مترتبة باش كل فكرة تبني على اللي قبلها.

المرحلة 1 • الأساس

الصفحات 3 حتى 8: شنو هو برنامج، API، backend، frontend، وكيفاش كتقرا الكود وكتلقى الملفات.

المرحلة 2 • التطبيق بلا ذكاء اصطناعي

الصفحات 9 حتى 14: قاعدة البيانات، تسجيل الدخول، الصلاحيات، الكاتالوغ والطلبات.

المرحلة 3 • الذكاء الاصطناعي

الصفحات 15 حتى 22: RAG، LLM، الأدوات، التأكيد، الحماية، SSE والسجلات.

المرحلة 4 • الربط والتشغيل

الصفحات 23 حتى 26: Docker، n8n، MCP، النشر والاختبارات.

المرحلة 5 • المناقشة

الصفحات 27 حتى 31: رحلة كاملة، الحدود الحالية، أسئلة اللجنة، خطة المراجعة وأجوبة التمارين.

كيفاش تعرف أنك فهمتي؟

إلا قدرت تجاوب على هادو بلا حفظ: شكون كيرسل الطلب؟ شكون كيتحقق؟ فين كتتخزن المعلومة؟ شنو يقع إلا فشل جزء؟ راه بديتي كتفهم المشروع فعلاً.

1 • شنو هو البرنامج؟

البرنامج هو تعليمات مكتوبة للحاسوب باش يدير خدمة.

Variable • متغير

بحال علبة مسمية كتحتافظ بقيمة. مثلاً stock فيه العدد المتوفر. إلا كانت القيمة 5 وبعنا 2، القيمة الجديدة تولي 3.

Function • دالة

مجموعة تعليمات عندها اسم. كتستقبل معلومات، كتدير خدمة، وقد ترجع نتيجة. checkStock كتقلب على المخزون ديال منتوج.

Array و Object

Array كيجمع خصائص ديال حاجة وحدة: منتوج عندو اسم وثمن ومخزون Object هي لائحة: مثلاً لائحة المنتجات اللي فالطلب.

Condition • شرط

if كتعني: إلا تحقق الشرط، دير هاد الخدمة. إلا كانت الكمية المطلوبة أكبر من المخزون، نرفض الطلب.

Folder و File

الملف فيه كود أو وثيقة. المجلد كيجمع ملفات مرتبطة. تقسيم المشروع كيساعدنا نلقاو المسؤولية ديال كل جزء.

سؤال 1

إلا stock فيه 5 والزبون طلب 7، شنو خاص البرنامج يدير؟ علاش؟

2 • ثلاث طبقات: بحال محل

تخيل متجر: الواجهة اللي كيشوف الزبون، الموظف اللي كيسير، والدفتر اللي فيه البيانات.

Frontend • الواجهة

هو اللي كيبان فالتليفون: أزرار، صور، فورموليرات، شات. فمشروعك هو mobile باستعمال React Native وExpo. كيجمع اختيار الزبون وكيعرض النتيجة.

Backend • الخادم

هو البرنامج اللي كيستقبل الطلبات، كيتحقق من الهوية والحقوق، كيحسب وكيقرا ويكتب فالقاعدة. فمشروعك هو backend باستعمال Node.js وExpress.

Database • قاعدة البيانات

هي التخزين المنظم والدائم: مستخدمين، منتجات، مخزون، طلبات ورسائل. PostgreSQL هي قاعدة البيانات ديالك.

```
Téléphone → API Express → PostgreSQL
Téléphone ← Réponse JSON ← API Express
```

علاش ما يخدمش التليفون مباشرة مع القاعدة؟

حيت المستخدم يقدر يبدل الطلبات اللي كيرسل. قواعد الأمان والثمن والمخزون خاصها تبقى فالخادم. إخفاء زر فالواجهة بوحدو ما كيحميش البيانات.

سؤال 2

إلا الزبون بدل ثمن المنتوج فالتليفون، واش الخادم خاصو يقبل داك الثمن؟

mobile/app • backend/src/app.ts • backend/src/config/database.ts

3 • شنو كيدير كل اسم تقني؟

ما تحفظش الأسماء بوحدها. ربط كل اسم بالخدمة ديالو.

TypeScript و JavaScript

JavaScript لغة البرمجة. TypeScript كتزيد وصف أنواع البيانات وكتكشف أخطاء أثناء التحليل أو البناء. ولكن البيانات اللي جاية من الشبكة خاصها حتى تحقق وقت التشغيل.

Express و Node.js

Express كينظم المسارات: مثلاً شنو نديرو ملي يجي طلب ديال المنتجات. JavaScript يخدم فالخادم Node.js كيخلي كود.

Expo و React Native

Expo كيبني واجهة الموبايل بمكونات React Native. كيوفر أدوات التشغيل والبناء وخدمات للموبايل. Expo Router كيربط الملفات بالشاشات والتنقل.

SecureStore و Axios و Zustand

Zustand كينظم حالة مشتركة فالواجهة، بحال المستخدم الحالي. Axios كيرسل طلبات HTTP. SecureStore كيتستعمل لتخزين حساس فالموبايل؛ ماشي قاعدة بيانات المنتجات.

Zod و Sequelize

Sequelize هو ORM: كيربط كود TypeScript بجداول SQL. Zod كيتأكد أن المدخلات عندها الشكل ما كتوافقش AGENTS.md؛ الإشارة القديمة لـSequelize فـPrisma والقيم المقبولة. الكود الحالي كيستعمل هاد التنفيذ.

pgvector و PostgreSQL

PostgreSQL كتحفظ البيانات. pgvector إضافة كتخزن وتقارن المتجهات العددية المستعملة فالبحث الدلالي.

backend/package.json • mobile/package.json • backend/src/models

4 • كيفاش كيهضر التليفون مع الخادم؟

API هي واجهة متفق عليها بين برنامجين. HTTP هو البروتوكول المستعمل هنا.

Request • الطلب

فيه العنوان، الطريقة، headers وممكن body. العنوان كيحدد الخدمة. headers فيها معلومات بحال نوع المحتوى ووسيلة إثبات الهوية.

Methods • الطرق

GET للقراية. POST للإرسال أو الإنشاء. PUT/PATCH للتعديل حسب المسار. DELETE للحذف. كل مسار عندو قواعدو.

JSON • شكل البيانات

صيغة نصية كتجمع مفاتيح وقيم. مثال تعليمي ديال منتوج: الاسم والثمن والمخزون. JSON ماشي قاعدة بيانات وماشي لغة ذكاء اصطناعي.

```
GET /api/products
Authorization: Bearer <access-token>

{"name":"T-shirt","price":120,"stock":5}
```

Response • الجواب

200 نجاح؛ 201 إنشاء؛ 400 مدخلات غير مقبولة؛ 401 خاص المصادقة أو التوكن غير صالح؛ 403 ما عندكش الحق؛ 404 ما تلقاش المورد؛ 429 طلبات كثيرة؛ 500 خطأ فالخادم. الشكل الدقيق ديال الجواب كيختلف حسب المسار.

سؤال 3

شنو الفرق بين 401 و403؟ شرح بمثال ديال زبون ومسؤول.

backend/src/app.ts • backend/src/routes

5 • كيفاش تقرا سطر كود؟

هاد المثال مبسط للتعليم، ماشي نسخ حرفي من المشروع.

```
const quantity = 2;
const product = await Product.findOne({
  where: { id: productId, storeId }
});
if (!product) throw new Error("Not found");
if (product.stock < quantity) {
  throw new Error("Insufficient stock");
}
```

const

كنسمي قيمة باش نرجع نستعملها. quantity هي الكمية المطلوبة.

await

كنستنا نتيجة خدمة غير فورية، بحال القراية من قاعدة البيانات. كتستعمل داخل دالة async. ما كتقصدش نحبس الخادم كامل.

where و findOne

قلب على سجل واحد كيوافق الشروط: معرف المنتوج ومعرف المتجر. هادي طريقة ORM؛ من تحت كاين استعلام SQL.

if و throw

كنختابرو واش المنتوج موجود وواش المخزون كافي. throw كيرسل خطأ لطبقة معالجة الأخطاء. فعملية شراء حقيقية كنحتاجو حتى transaction وlock.

طريقة القراية

بدا بالمدخلات، من بعد الشروط، من بعد الوصول للقاعدة، ومن بعد النتيجة. ما تحاولش تفهم جميع imports قبل ما تعرف الهدف ديال الدالة.

6 • فين تلقى كل حاجة؟

كل مرة كتشرح ميزة، حاول تورّي الملف المسؤول عليها.

backend/src/app.ts و server.ts

app كيركب middlewares والمسارات. server كيربط مع قاعدة البيانات وكيبدا يسمع للطلبات.

backend/src/routes

فيه auth وproducts وcategories وorders وdocuments وRoute. agent هي نقطة دخول HTTP، كتستقبل وتوجه الخدمة.

backend/src/models و schemas

models كتوصف السجلات والعلاقات. schemas كتوصف المدخلات المقبولة. Model ديال Product ماشي هو schema ديال طلب إنشاء Product.

backend/src/services

orderConfirmation للتأكيد؛ agentTools للأدوات؛ llm للتواصل مع النموذج؛ rag للبحث فالوثائق؛ guardrails للحماية؛ mcpShipmentClient للتتبع الخارجي.

mobile/app و mobile/src

app تخزين الحالة، المصادقة والمكونات المشتركة. ملفات API، فيه اتصال src. فيه الشاشات والتنقل (tabs) كتنظم تبويبات الواجهة.

migrations و docs و stitch_shopmate

migrations كتغير بنية القاعدة. docs فيها وثائق المشروع. stitch_shopmate مرجع التصميم؛ التصميم المرئي بوحدو ما كيبرهنش أن الخدمة مربوطة بالخادم.

7 • قاعدة البيانات: شنو كتخزن؟

Table جدول، Row سجل، Column خاصية، ID معرف كيخلينا نميزو السجل.

Store / User / Customer

Store هو المتجر. User حساب الدخول والأدوار. Customer بيانات الزبون المستعملة فالطلبات. ماشي بالضرورة نفس المعرف Customerو User فالمسارات كاين ربط بالمتجر والبريد؛

Category / Product

Category كتجمع المنتجات فصنف. Product فيه بيانات المنتوج بحال الثمن والمخزون ومعرف المتجر والصنف. imageUrl رابط صورة، ماشي بالضرورة خدمة رفع وتخزين الصور.

Order / OrderItem

Order رأس الطلب: صاحبو والحالة والمجموع. OrderItem سطر فيه المنتوج والكمية وثمن الوحدة وقت الشراء. طلب واحد يقدر يكون فيه بزاف ديال السطور.

Conversation / Message / AgentLog

Conversation محادثة، Message رسالة داخلها. AgentLog أثر تقني لتفاعل: الأدوات والمصادر والمدة والاستهلاك والنتيجة.

Document / Chunk / RefreshToken

Document وثيقة معرفة. Chunk جزء منها مع تمثيل عددي. RefreshToken سجل لتجديد الدخول؛ القيمة المخزنة هي hash ديال التوكن.

```
Store → Products / Customers / Orders
Customer → Orders → OrderItems → Product
Conversation → Messages
Document → Chunks
```

backend/src/models/index.ts • backend/src/models/*.ts

8 • التسجيل والدخول

Authentication كتجاوب: شكون نتا؟ Authorization كتجاوب: شنو مسموح ليك تدير؟

التسجيل

الواجهة كترسل المعلومات. الخادم كيتحقق من الشكل. كيحسب hash لكلمة السر باستعمال bcrypt وكيحفظ الحساب. الحساب العادي كيتسجل بدور CUSTOMER.

ماشي تشفير قابل للعكس Hash

الخادم ما خاصوش يرجع كلمة السر الأصلية من hash. وقت الدخول bcrypt كيقارن الكلمة المدخلة مع hash المخزن بطريقة مناسبة.

الدخول

الخادم كيلقى الحساب وكيقارن كلمة السر. إلا نجح التحقق، كيرجع access token وrefresh token. إلا فشل، ما كيعطيش صلاحية الدخول.

بطاقة موقعة • JWT

Access token كيحمل معلومات بحال userId وstoreId وrole. التوقيع كيخلي الخادم يكشف التلاعب. الموقع ماشي بالضرورة مشفر؛ ما نحطوش فيه أسرار JWT.

```
Login → Vérification bcrypt → Tokens
Requête → Vérification JWT → Contrôle du rôle → Service
```

سؤال 4

علاش ما نحفظوش كلمة السر واضحة فالقاعدة؟ وعلاش ما نثقوش فـrole جاية من body؟

backend/src/routes/auth.ts • backend/src/config/jwt.ts • backend/src/middlewares/auth.ts

9 • Refresh rotation و logout

Access token للاستعمال اليومي القصير، وrefresh token لتجديد الدخول بلا إعادة كلمة السر كل مرة.

عندما كيسالي access token

الواجهة كتلقى 401، وكتطلب refresh. الخادم كيتحقق من refresh token ومن الصلاحية ديالو، وكيصدر زوج جديد ديال التوكنات.

Rotation • التبديل

القديم كيتحيد وكيخرج واحد جديد. الهدف هو ما يبقاش نفس السر صالح للتجديد طول الوقت. Refresh فالقاعدة كيكون hash بدل القيمة الأصلية.

Axios mutex

إلا عدة طلبات جابو 401 فنفس الوقت، آلية فالعميل كتنسق التجديد باش ما يبداوش كاملين refresh مستقل فنفس اللحظة.

Logout • الخروج

الواجهة كتحيد معلومات الدخول والخادم كيلغي refresh. ولكن access JWT اللي خرج من قبل يقدر يبقى صالح حتى يسالي الوقت ديالو إلا ما كانتش آلية إبطال إضافية.

حد خاص تعرفو

الـrotation الحالية ما خاصناش نوصفوها بضمان مطلق ضد جميع الطلبات المتزامنة. حذف القديم وإصدار الجديد خاصهم مراجعة التزامن والمعاملات باش يكون الضمان أقوى.

سؤال 5

واش logout كيعني آلياً أن أي access token قديم ولى باطل فوراً؟

backend/src/routes/auth.ts • mobile/src/api/client.ts • mobile/src/auth

10 • Multi-tenant: كل متجر وبياناتو

Tenant هنا هو متجر. نفس التطبيق يقدر يخدم أكثر من متجر مع عزل البيانات.

storeId

معرف المتجر كيدخل فشروط الوصول للبيانات التابعة ليه. فالمسارات المحمية كيجي من JWT المتحقق منو فالخادم، ماشي من اختيار الزبون فالطلب.

مثال

متجر A عندو منتوج، ومتجر B عندو منتوج آخر. إلا مستخدم A عرف ID ديال منتوج B، الخادم خاصو يبحث بالـID وstoreId معاً. معرفة المعرف ما كتمنحش الحق.

الزبون داخل نفس المتجر

حتى داخل A، ما خاصش زبون يشوف طلب زبون آخر. كنحتاجو التحقق من مالك الطلب زيادة على المتجر.

الدور ADMIN

إظهار شاشة الإدارة غير للمسؤول مفيد فالواجهة. لكن الحماية الحقيقية كتكون حتى فالخادم قبل القراءة أو التعديل.

تفصيل فنسخة الديمو

التسجيل والدخول كيرتبطو بمتجر الديمو المحدد فالخادم. بعض قراءات الكاتالوغ العامة كتستعمل متجر الديمو كذلك. ما تقولش إن كل request عامة كتحتاج JWT، ولا إن التسجيل كيدير اختيار حر للمتجر.

سؤال 6

علاش الفلترة بالـorderId بوحدو ما كافياش لحماية طلبات الزبناء؟

backend/src/middlewares/tenant.ts • backend/src/models/tenantScope.ts • backend/src/routes/orders.ts

11 • الكاتالوغ والإدارة

CRUD: Create إنشاء، Read قراية، Update تعديل، Delete حذف.

رحلة عرض المنتجات

الشاشة كتطلب المنتجات. الخادم كيتحقق من الفلاتر والمتجر. القاعدة كترجع النتائج. الواجهة كتبيّن اللائحة، التحميل، الفراغ أو الخطأ.

تقسيم اللائحة • Pagination

ما كنجيبووش جميع المنتجات دفعة وحدة. كنطلبو جزء، ومن بعد جزء آخر. هادشي كيخفف الاستجابة والذاكرة.

المسؤول كيعدل

يقدر يدير عمليات الكاتالوغ المسموحة: منتجات، أصناف، ثمن ومخزون. الخادم كيتحقق من الدور ADMIN والمدخلات. حذف منتوج خاصو واجهة تأكيد ومعالجة القيود المرتبطة بالطلبات.

التحقق • Validation

مثلاً الكمية عدد موجب والثمن قيمة مقبولة. TypeScript بوحدو ما كيكفيش، حيت الطلب الخارجي يقدر يكون بأي شكل. Zod كيتحقق وقت التشغيل.

واش التغيير كيبان مباشرة؟

القاعدة كتكون المرجع الجديد. المساعد اللي كيستعمل أدوات القراية كيلقى القيم الحالية. الشاشة المفتوحة قد تحتاج إعادة تحميل؛ هادشي ما كيعنيش كاين broadcast تلقائي لجميع الهواتف.

سؤال 7

إلا المسؤول بدل الثمن، واش خاصنا نعاودو ندربو LLM باش يعرفو؟

backend/src/routes/products.ts • backend/src/routes/categories.ts • mobile/app/(tabs)

12 • الطلب والمخزون و Transaction

بيع واحد خاصو يحدث الطلب والسطور والمخزون بشكل منسجم.

مثال بالحساب

المخزون 5، الزبون باغي 2، ثمن الوحدة 120. مجموع السطور 240 والمخزون من بعد 3. الثمن كيتقرا من القاعدة؛ ما كنخليوش الزبون يفرض المجموع.

Transaction • معاملة

كتجمع تغييرات مترابطة: إما تنجح كلها ويتدار commit، وإما يتدار rollback ويرجع أثر المعاملة كما كان. ما نبغيوش ينقص المخزون والطلب ما يتسجلش.

Lock • قفل على السجل

إلا جوج زبناء بغاو آخر قطعة فنفس الوقت، القفل كينظم تعديل السجل. العملية الثانية كتستنى وتعاود التحقق من المخزون المتاح.

OrderItem كيحفظ الثمن

إلا ثمن المنتوج تبدل غداً، خاص الطلب القديم يبقى عندو ثمن الشراء ديالو. لهذا كنحتاجو ثمن الوحدة فسطر الطلب.

التأكيد

المسار ديال الشات كيحضّر recap ومن بعد كينفذ عند التأكيد. المسار HTTP ديال الطلب عندو schemas وقواعد خاصة؛ ما نخلطوش بين الآليتين ونفترضو أنهما نفس الدالة.

سؤال 8

شنو المشكل إلا نقصنا المخزون ثم وقع خطأ قبل إنشاء الطلب بلا transaction؟

backend/src/routes/orders.ts • backend/src/services/orderConfirmation.ts • backend/src/models/OrderItem.ts

13 • LLM و Agent: شنو الفرق؟

LLM نموذج لغوي كيعالج النص وكيولد كلام. Agent هو النظام اللي كينظم استعمال هاد النموذج مع أدوات وقواعد.

ماشي قاعدة بيانات المتجر LLM

النموذج يقدر يعطي كلام مقنع حتى إلا ما عندوش المعلومة. ما عندوش تلقائياً المخزون الحقيقي ولا طلبات الزبناء.

دور Agent

الخادم كيجمع السؤال والتاريخ والسياق، كيطبق الحماية، كيطلب من النموذج، كيتحقق من الأدوات المقترحة، كينفذ المسموح وكيحضّر الجواب.

System prompt

تعليمات على الدور واللغة والحدود. مفيدة، ولكن ما كتعوضش التحقق البرمجي من الهوية والحقوق والمدخلات.

Provider

مزود خارجي للنموذج. الكود كيقبل إعدادات endpoint/model/key عبر البيئة. فالمناقشة قول المزود اللي ضبطتي فعلاً؛ ما تدعيش أن جميع المزودين مجربين.

Tokens

قطع من النص كيستعملها النموذج. ماشي دائماً كلمات كاملة. الاستهلاك فيه نص مرسل ونص مولد، وكيأثر على التكلفة والمدة.

سؤال 9

شكون كيقرر واش أداة معينة مسموحة وتتنفذ فعلاً: النموذج بوحدو ولا الخادم؟

backend/src/services/llm.ts • backend/src/routes/agent.ts

14 • RAG: نجبدو المعرفة قبل الجواب

RAG = Retrieval Augmented Generation. بالبساطة: نقلبو على معلومة مناسبة، ونمدوها للنموذج باش يجاوب منها.

المشكل

الزبون كيسول على مدة التوصيل. النموذج ما خاصوش يخترع سياسة المتجر. خاصو يعتمد على وثيقة حقيقية مرفوعة من المسؤول.

مرحلتان

هي البحث على المقاطع المناسبة وقت Retrieval. هي إدخال الوثائق وتحضيرها للبحث Ingestion السؤال. Generation هي صياغة الجواب من السياق.

رفع الوثيقة

المسؤول كيرفع ملف md أو txt حتى 2 Mo عبر POST /api/documents. الخادم كيحضّر المقاطع والمتجهات ويحفظها. واجهة الإدارة كتخدم لائحة الوثائق والحذف وحالة المعالجة؛ إلا فشل الرفع ما نفترضوش أن المعرفة تزادت.

مثال تعليمي

وثيقة كتقول: التوصيل للمدينة بين يومين وثلاثة. السؤال: شحال حتى توصلني؟ كنرجعو المقطع المناسب ونطلبو جواب مبني عليه مع المصدر.

```
Document → Chunks → Embeddings → pgvector
Question → Embedding → Recherche → Contexte → Réponse + source
```

واش هادي كتدرب النموذج؟

لا. فهاد المشروع كنضيفو سياق وقت الطلب، ما كنبدلوش أوزان النموذج. رفع وثيقة ماشي fine-tuning.

سؤال 10

شنو الفرق بين نرفع سياسة إرجاع جديدة وبين ندرب نموذج جديد؟

backend/src/services/rag.ts • backend/src/routes/documents.ts

15 • Chunk و Embedding و Similarité

هاد الكلمات كتشرح كيفاش البحث كيلقى معنى قريب حتى إلا الكلمات ماشي هي نفسها.

Chunk • مقطع

بدل ما نرسلو الوثيقة كاملة، كنقسموها لمقاطع. فالكود الهدف تقريباً 800 حرف مع overlap ديال 100 حرف. التداخل كيحافظ على سياق قريب من الحدود.

Embedding • تمثيل عددي

النص كيتحول للائحة أعداد كتستعمل لقياس القرب الدلالي. فالإعداد الافتراضي: text-embedding-3-small و1536 بُعد. ماشي لائحة ديال 1536 كلمة.

مثال للفهم فقط

تصور أن كل نص نقطة فخريطة. جمل قريبة فالمعنى كتكون أقرب حسب النموذج. الخريطة الحقيقية فيها أبعاد كثيرة؛ رسم بجوج أبعاد غير تشبيه.

HNSW و Cosine

كيسرع البحث على جيران قريبين بطريقة تقريبية. HNSW index. مقياس مرتبط باتجاه المتجهات Cosine. ما كيولدش الجواب، غير كيساعد فالاسترجاع.

Threshold • العتبة

الإعداد الافتراضي فالبحث 0.75 وtopK ديال 6. إلا ما كاينش سياق مناسب، الجواب المعرفي يكون: Je n'ai pas cette information. 0.75 ماشي ضمان صحة بنسبة 75%.

سؤال 11

علاش خاص سؤال المستخدم والوثائق يستعملو embeddings متوافقين؟

backend/src/services/rag.ts • backend/sql

16 • RAG ولا Tools؟

هاد الفرق هو قلب المشروع، وغالباً غادي تسولك عليه اللجنة.

المعرفة الثابتة نسبياً

سياسة الإرجاع، شروط التوصيل، معلومات المتجر: كنبحثو عليها فالوثائق باستعمال RAG ونبينو المصدر.

البيانات الحية

الثمن والمخزون وحالة الطلب: أدوات كتقرا من قاعدة البيانات أو الخدمة المخصصة. ما نعتمدوش على وثيقة قديمة باش نقولو شحال بقا فالمخزون.

الأفعال

إنشاء أو إلغاء طلب: أدوات كتجهز العملية وكتطلب التأكيد، ومن بعد التنفيذ المحمي. الجواب النصي بوحدو ما كيخلقش طلب.

أمثلة

«شنو شروط الإرجاع؟» → RAG.

«واش باقي التيشورت؟» → checkStock.

«بغيت جوج» → createOrder ثم recap ثم تأكيد.

«فين وصلات؟» → getShipmentStatus بعد التحقق من الملكية.

الأدوات ماشي غير كتابة

أداة ولكن ما كتبدل والو فالقاعدة. الفرق searchProducts. كيشمل حتى القراية Function Calling العملي هو مصدر المعلومة وطبيعة الخدمة.

سؤال 12

علاش ما نحطوش الثمن فوثيقة RAG ونخليو المساعد يعتمد عليه دائماً؟

backend/src/services/agentTools.ts • backend/src/services/rag.ts

17 • الأدوات الثمانية وكيفاش كتتنفذ

النموذج كيقترح اسم أداة وarguments. الخادم كيتحقق قبل التنفيذ.

أدوات الكاتالوغ

كتقرا checkStock. كتجيب تفاصيل منتوج getProductDetails. كتقلب على المنتجات searchProducts التوفر والمعطيات الحية المرتبطة به.

أدوات الطلبات

كتجيب حالة طلب getOrderStatus. كتبدأ مسار إنشاء طلب بالتأكيد createOrder. كتجيب طلبات الزبون listMyOrders. كتبدأ مسار الإلغاء بالتأكيد cancelOrder.

أداة التتبع

كتطلب حالة الشحنة عبر MCP getShipmentStatus بعد التحقق من الطلب والمتجر والزبون.

```
{"name":"checkStock","arguments":{"productId":42}}
```

سلسلة التحقق

اسم داخل allowlist → قراءة JSON → تحقق Zod → حقن هوية المستخدم والمتجر من الخادم → تنفيذ الخدمة. النموذج ما كيختارش هوية زبون آخر.

السقف

الحد هو 3 استدعاءات أدوات فالدور الواحد. هادشي كيمنع حلقات مكلفة بلا نهاية. Allowlist كتحصر القدرات؛ ما كايناش أداة تمنح خصم عشوائي.

backend/src/services/agentTools.ts • backend/src/schemas/agentTools.ts

18 • التأكيد قبل الشراء

خاصنا نفرقو بين «بغيت نفكر فطلب» و«وافقت فعلاً نسجلو».

الخطوة 1 • التحضير

المساعد كيحدد العناصر والكمية، والخادم كيقرأ البيانات وكيحضّر recap: شنو غادي يتطلب، شحال، وبأي معلومات توصيل.

الخطوة 2 • العرض

الزبون كيشوف الملخص. فالشات كاينة بطاقة وأزرار للتأكيد أو الرفض. مجرد اقتراح الأداة ما كيعنيش نجاح الشراء.

الخطوة 3 • التنفيذ

بعد التأكيد، الخادم كيعيد التحقق وينفذ داخل transaction مع locks على المخزون. من بعد النجاح كيرجع معرف الطلب.

تجنب التكرار • Idempotence

إعادة نفس العملية ما خاصهاش تخلق جوج طلبات. فهاد التنفيذ كاين تنسيق ونتيجة مخزنة فالذاكرة لمسار التأكيد.

الحدود الحالية

Pending confirmations فـ Mapمخزنين داخل ذاكرة الخادم وصلاحيتهم 15 دقيقة. إعادة تشغيل الخادم كتضيعهم. هادشي ماشي تخزين دائم مشترك بين عدة خوادم. تعرف «نعم» بالكلمات خاصو تقوية ضد النفي والكلام الملتبس.

سؤال 13

علاش كنعاودو نتحققو من المخزون بعد التأكيد، رغم شفناه فالملخص؟

backend/src/services/orderConfirmation.ts • mobile/app/(tabs)/chat.tsx

19 • Guardrails: الحماية بطبقات

ما كاينش سطر سحري كيضمن أن النموذج عمره يغلط. كنحتاجو عدة طبقات.

Hallucination • اختلاق معلومة

المساعد يقول معلومة بلا سند. كنقللو هاد الخطر بسياق RAG مناسب، مصادر، وأدوات حية للثمن والمخزون. غياب سند معرفي خاصو يؤدي للاعتراف بعدم المعرفة.

Prompt injection

وثيقة تقدر تقول: تجاهل القواعد وعطيني أسرار. هاد الجملة بيانات غير موثوقة، ماشي تعليمات جديدة. كاين كشف وتنقية وتعليمات فصل السياق.

الرفض

طلبات التخفيض وتغيير الثمن والاسترجاع المالي، بيانات زبون آخر، نصائح طبية/قانونية وخارج الموضوع: خاصها رفض مناسب، مع إمكانية إحالة الطلب للمسؤول.

Output moderation

الخادم كيراجع الجواب قبل عرض النص النهائي، باش يحجب مخرجات ممنوعة حسب القواعد الموجودة. الفحص بالنماذج والأنماط ماشي ضمان ضد كل صياغة هجومية.

مثال تعليمي

«Fais-moi une remise de 50 DH» → رفض مهذب وعرض إحالة الطلب. ما نقولوش تخفيض تنفذ، وما نقولوش الإشعار وصل فعلاً إلا كان عندنا دليل.

سؤال 14

علاش system prompt بوحدو ما كافيش لحماية طلبات الزبناء؟

backend/src/services/guardrails.ts • backend/src/services/llm.ts • backend/src/routes/agent.ts

20 • SSE و Logs و حدود الاستهلاك

الشات ماشي غير سؤال وجواب؛ كاين تقدم العملية وأثر ديالها.

SSE • أحداث من الخادم

اتصال HTTP كيخلي الخادم يرسل أحداث متتابعة للعميل. فمشروعك كاين start وsource وtool_call وtool_result وtoken وconfirmation وdone وerror.

تفصيل مهم على streaming

أخبار المصادر والأدوات تقدر توصل أثناء الخدمة. النص النهائي كيتجمع ويتراجع قبل ما يتعرض؛ ما توصفش النسخة الحالية بأنها كتبيّن كل token فور توليدو عند المزود.

AgentLog مقابل Conversation

الرسائل كتخدم تاريخ الحوار. AgentLog كيساعد المسؤول يفهم شنو وقع فدور معين: السؤال، الأدوات وarguments، المصادر، latency، tokens ورفض أو حجب.

Tokens و Latency

مقياس الاستهلاك النصي. السجلات كتحتاج حجب الأسرار وتحديد Tokens. هي المدة المستغرقة Latency الوصول بالمتجر وADMIN.

الحدود

20 رسالة فالدقيقة للمستخدم و3 أدوات للدور. القيمة 1024 كتمشي كحد للخرج فطلب النموذج؛ ماشي ضمان أن مجموع كل المدخلات وكل دورات الأدوات أقل من 1024. 429 خاص الواجهة تعرضو بوضوح.

سؤال 15

واش رسالة «جاري البحث» فالواجهة كتعني أن الجواب النهائي تراجع ووجد؟

backend/src/routes/agent.ts • backend/src/middlewares/rateLimiter.ts • mobile/src/api/agentChat.ts

21 • MCP: تتبع الشحنة

MCP = Model Context Protocol. طريقة منظمة باش برنامج يتواصل مع أدوات كيقدمها خادم آخر.

الأجزاء

عندك خادم API الرئيسي، client ديال MCP فالـbackend، وخادم MCP منفصل كيعرض getShipmentStatus.

الرحلة

الزبون يسول فين وصل الطلب. API كيتحقق أن الطلب ديالو ومن متجره، من بعد client كيتصل بـMCP_SERVER_URL. النتيجة كتترجع للمساعد باش يشرحها.

```
Client → Agent → Vérification de propriété
Agent → MCP client → MCP server → Statut simulé
Agent → Réponse au client
```

بيانات محاكاة • Mock

فالديمو بيانات النقل محاكاة. هادشي كيبين الربط والبروتوكول، ولكن ما كيثبتش أن شركة توصيل حقيقية مرتبطة.

fallback و Timeout

العميل عندو مهلة 5 ثوان وكيحقق فالجواب. إلا الخدمة ما جاوباتش، يرجع رسالة مفهومة بدل تفاصيل داخلية أو حالة شحنة مخترعة.

سؤال 16

شنو الفرق بين getOrderStatus من قاعدة المتجر وgetShipmentStatus من خدمة التتبع؟

backend/src/mcp/shipmentServer.ts • backend/src/services/mcpShipmentClient.ts

22 • n8n و Webhook

n8n كينظم أتمتة خارج التطبيق. Webhook طلب HTTP كيخبر خدمة أخرى أن حدث وقع.

الحدث

تخلق طلب أو المساعد أحال طلب للمسؤول. Backend كيجهز تفاصيل الحدث وكيبعثها للرابط المضبوط فالإعدادات.

سلسلة خطوات • Workflow

يستقبل → يقرأ نوع الحدث → يحضر نص الإشعار → قناة إشعار للمسؤول. إعداد القناة Webhook والمفاتيح كيبقى مطلوباً خارج الكود.

```
Order created → Backend webhook → n8n → Notification
```

علاش نفصلوه؟

قواعد الشراء تبقى فالخادم. الإشعارات يمكن تبديل قناتها وإعدادها فالأتمتة بلا خلطها مع حساب المخزون.

حدود الديمو

وجود ملف workflow ما كيعنيش أنه منشور ومفعل. فشل الإشعار ما خاصوش يتحول لوعد أن المسؤول توصل. خاص تجربة حقيقية وإثبات وصول الإشعار.

سؤال 17

إلا n8n كان طافي، واش هادشي بالضرورة كيعني الطلب ما تسجلش؟

backend/src/services/n8nNotifications.ts • backend/n8n

23 • Docker و migrations و seed

الهدف هو تجهيز تشغيل متشابه فالأجهزة والخادم.

Image و Container

عملية تشغيل معزولة نسبياً انطلقت من Container. وصف مبني فيه التطبيق وما يحتاجو Image container. Volume كيحتافظ ببيانات القاعدة خارج عمر image.

Multi-stage Dockerfile

مرحلة build كتجهز الكود، ومرحلة runtime كتحتافظ بما يلزم للتشغيل. مستخدم non-root كيقلل الصلاحيات داخل الحاوية.

Compose

كيجمع API وPostgreSQL مع pgvector وشبكة وإعدادات وفحوص صحة. داخل حاوية API، اسم خدمة DB هو اللي كيشير للقاعدة؛ localhost كيعود لنفس الحاوية.

Migration

تغيير منظم فبنية القاعدة: جدول جديد أو عمود أو index. ماشي نسخة احتياطية. pgvector SQL كيمكن الامتداد والبنية اللازمة للبحث المتجهي حسب السكريبتات.

Seed

كيضيف بيانات البداية أو الديمو. تشغيله فالإنتاج خاص يكون مقصود ومراجع؛ ماشي كل إعادة تشغيل خاصها تعاود تزيد بيانات تجريبية.

Environment

الإعدادات بحال DB host ومفاتيح API كتجي من البيئة. env. ما يتنشرش فـ.env.example. git فيه أسماء وقيم وهمية. ما تحطش مفتاح المزود السري فـEXPO_PUBLIC.

backend/Dockerfile • docker-compose.yml • backend/migrations • backend/src/seed.ts

24 • النشر والاختبارات

«الكود موجود» و«جربناه فالإنتاج» جوج درجات مختلفين ديال الدليل.

Deployment • النشر

تشغيل API وقاعدة البيانات فخدمة مستضافة، إعداد secrets وHTTPS وتشغيل migrations. الموبايل خاصو عنوان backend المنشور فإعداداته.

health و CORS

CORS كينظم طلبات المتصفح من origins تطبيق وحدو وما كيحميش ؛ ماشي بديل للمصادقة native. health endpoint كيبين واش الخدمة كتجاوب، وماشي بالضرورة فحص لكل اعتماد خارجي.

Unit test

كيختبر جزء صغير: مثلاً schema كيرفض كمية سالبة. Integration test كيختبر أجزاء مرتبطة. E2E كيدوز رحلة مستخدم كاملة.

Mock

بديل مبرمج لخدمة حقيقية أثناء الاختبار. مفيد باش نختبرو منطق محدد بسرعة؛ ما كيبرهنش أن PostgreSQL الحقيقي أو الشبكة خدامين.

شنو كاين هنا؟

backend كيبني TypeScript وكيشغل سكريبتات اختبارات للمصادقة والعزل والأدوات فال npm test ولا تحققنا من هاتف suite فهاد جلسة إعداد الدليل ما شغلناش n8n. والتأكيد والحماية والتكلفة و حقيقي.

سؤال 18

واش test ناجح باستعمال mock كافي باش نقولو إن البيع المتزامن فقاعدة الإنتاج مضمون؟

backend/package.json • backend/src/scripts • mobile/src/config/api.ts

25 • دوز الرحلة كاملة بصوتك

هادي حكاية المشروع اللي خاصك تقدر تشرحها فالسبورة.

1 • دخول

الزبون يسجل الدخول. API كيتحقق من كلمة السر وكيصدر التوكنات. الواجهة كتستعمل access token فالطلبات المحمية.

2 • سؤال معرفي

يسول على الإرجاع. API يسترجع مقاطع وثائق المتجر المناسبة، ويطلب جواب مبني عليها. إلا ما لقا سند مناسب، يصرح بعدم المعرفة.

3 • منتوج

يسول على تيشورت. النموذج يقترح searchProducts أو checkStock. الخادم يتحقق من الأداة وargumentsويقرا البيانات الحية من متجر المستخدم.

4 • شراء

يبغي جوج. كيتعرض recap. يوافق. الخادم يعاود يتحقق ويكتب الطلب والسطور وينقص المخزون داخل transaction.

5 • من بعد

الواجهة كتبين رقم الطلب. إشعار n8n ممكن يتبعث إذا الربط مهيأ. التتبع عبر MCP كيرجع حالة محاكاة فالديمو. ADMIN يراجع أثر الشات فـAgentLog.

التمرين الكبير

رسم هاد الرحلة بوحدك وسمّي فين كنستعملو JWT وRAG وTools وtransaction وMCP. إلا قدرت تفسر علاش كل واحد كاين، راه فهمتي العمود الفقري للمشروع.

26 • شنو تقول بصراحة على الحدود؟

الفهم القوي كيبان حتى ملي كتفرق بين الهدف، التنفيذ الحالي، والدليل اللي عندك.

ما تقولش: أمان مطلق

قول: عندي طبقات تحقق من الهوية والمتجر والملكية والمدخلات، ومعاها guardrails واختبارات. الكشف بالأنماط ما كيضمنش صد جميع الهجمات.

ما تقولش: logout كيلغي كلشي فوراً

قول: كنبطل refresh ونحيد التوكنات من العميل؛ access JWT قد يبقى صالح حتى نهاية المدة.

ما تقولش: تأكيد دائم وموزع

قول: تأكيد الشات مخزن فالذاكرة حالياً بمهلة، ونحتاج تخزين مشترك ودائم باش نقوي التشغيل بعد restart أو مع عدة instances.

ما تقولش: 75% احتمال صحة

قول: هادي عتبة similarity فالبحث، خاص معايرتها بأسئلة ممثلة ومراجعة الأجوبة والمصادر.

ما تقولش: كل التكاملات حقيقية

قول: MCP كيرجع بيانات محاكاة، وn8n خاصو إعداد وتجربة، والنشر والهاتف كيحتاجو أدلة تشغيل مستقلة.

README والكود

فيه بعض عبارات أقوى من الضمان الحالي، بحال الإبطال الفوري أو منع التكرار بشكل مطلق. README شرحك يتبع التنفيذ وحدوده، وتحديث التوثيق يكون فمهمة منفصلة.

27 • أسئلة اللجنة: جواب وفهم

هادو صيغ قصيرة بالفرنسية. فهم المعنى بالدارجة قبل ما تستعملهم.

Présente ton projet.

ShopMate est une application mobile e-commerce COD avec un assistant. Il répond à partir des documents de la boutique et utilise des outils contrôlés pour les données et les commandes.

المعنى: تطبيق بيع بمساعد عندو مصادر وحدود، ماشي شات عام.

Pourquoi RAG et function calling ?

Le RAG fournit le contexte documentaire. Les outils lisent les données actuelles et exécutent les opérations autorisées.

المعنى: سياسة التوصيل من الوثيقة، المخزون من القاعدة، والشراء عملية محمية.

Comment isoles-tu les boutiques ?

Le serveur utilise le storeId de l'identité authentifiée et filtre les accès. Pour les commandes, il vérifie aussi le client propriétaire.

المعنى: معرف المتجر بوحدو ما كافيش لملكية الطلب.

Comment évites-tu une commande involontaire ?

Le chat prépare un récapitulatif. Après confirmation, le serveur vérifie les données et exécute la transaction.

المعنى: نية الشراء ماشي هي الموافقة النهائية.

As-tu utilisé l'IA pour coder ?

Oui. Je vérifie le code produit, je documente les corrections et je dois pouvoir expliquer les choix et les limites.

ما تدعيش تحقق ما درتيهش. ورّي مثال حقيقي من journal ديالك.

28 • خطة يومين بلا تشتت

المناقشة اللي قلتي نهار 22 أكتوبر. الموعد العام فالوثائق 24 أكتوبر؛ حضر على أساس الأقرب.

20 أكتوبر • الأساس

ساعة: الصفحات 3 حتى 10 مع رسم المعمارية. راحة. ساعة: الكاتالوغ والطلبات والمعاملات. نصف ساعة: حل الأسئلة بصوتك بلا دليل.

21 أكتوبر • الذكاء الاصطناعي

ساعة ونصف: RAG والأدوات والتأكيد. راحة. ساعة: الحماية وSSE وMCP وDocker. سجل شرح صوتي قصير وسمع فين كتخلط.

21 أكتوبر • ديمو

راجع رحلة الدخول، البحث، سؤال وثيقة، recap، رفض ثم تأكيد، عرض الطلب، ثم الإدارة والوثائق والسجلات. استعمل بيانات ديمو وخلي نتيجة كل خطوة واضحة.

بديل الديمو

وجد تسجيل فيديو ولقطات من تجربة فعلية إلا الشبكة تعطل. إلا عرضتي تسجيلاً، قول بأنه تسجيل؛ ما تقدموش كتجربة مباشرة.

22 أكتوبر • مراجعة خفيفة

كرر تعريف المشروع والرحلة الكاملة وخمس نقاط: RAG، storeId، JWT مقابل الأدوات، التأكيد، transaction. إلا ما عرفتيش تفصيل، قول فين غادي تقلب عليه.

ديالك Journal

كتب بكلامك: شنو طلبت فهاد الجلسة، شنو فهمت، شنو الحدود اللي اكتشفت، وشنو باقي ما واضحش. ما تسجلش نتيجة اختبار ما شغلتوش.

29 • أجوبة التمارين ومفتاح المراجعة

حاول تجاوب قبل ما تقرا. الهدف تفهم السبب، ماشي تحفظ نفس الجملة.

1 حتى 3

1: نرفض كمية 7 لأن المتوفر 5. 2: لا؛ الثمن المرجعي من الخادم والقاعدة. 3: 401 هوية ناقصة أو غير صالحة؛ 403 هوية معروفة ولكن بلا الصلاحية المطلوبة.

4 حتى 6

4: hash و الموقع قد يبقى صالح access، فالطلب يقدر يتزور. 5: لا roleكيقلل ضرر تسرب كلمات السر، حتى انتهاء المدة. 6: خاص حتى المتجر وصاحب الطلب.

7 حتى 9

7: لا، الأداة كتقرا الثمن الحالي. 8: المخزون ينقص بلا طلب؛ transaction كتجنب الأثر الجزئي داخلها. 9: الخادم هو اللي يتحقق وينفذ، النموذج كيقترح.

10 حتى 12

10: RAG كيضيف سياق وقت السؤال، التدريب كيغير النموذج. 11: باش المقارنة العددية تكون عندها معنى وأبعاد متوافقة. 12: الثمن فالوثيقة يقدر يولي قديم.

13 حتى 15

13: المخزون يقدر يتبدل بين الملخص والتأكيد. 14: الحقوق خاصها تحقق برمجي مستقل عن النص. 15: لا، حدث التقدم ماشي الجواب النهائي المراجع.

16 حتى 18

16: حالة الطلب الداخلية مقابل تتبع الشحنة بالخدمة الخارجية. 17: لا بالضرورة؛ الإشعار منفصل عن تسجيل الطلب. 18: لا؛ خاص تجربة مناسبة بقاعدة حقيقية وتزامن.

من بعد هاد الدليل

فتح ملف واحد، اختار دالة وحدة، وشرح: المدخلات، التحقق، القراءة أو الكتابة، النتيجة، وحالة الفشل. هادي هي الطريقة باش توصل للفهم التفصيلي سطر بسطر.